Drishti दृष्टि

# Drishti VaR engine
# Architecture, data contract and how VaR works

Any set of trades or positions, in any reporting currency

*See what your data hasn't told you yet.*

ANY DATA. ANY DOMAIN. ONE GRAMMAR.

**Ashutosh Sinha**

October 2026

Historical simulation over 509 scenario days, window and confidence configurable

IN THIS DECK

1–4 Why, how it works, data in, computed
5 Scale, capacity, concurrency
6–7 Storage, operations, model risk
8 Editions: Risk, Market, World
9–10 Regulatory reporting, roadmap
A Appendix: reference

TL;DR

# Executive summary

| | |
|---|---|
| **What is it?** | A VaR engine in the Drishti server: historical simulation over 509 scenario days (or a set window), VaR 99% (95% by setting) and ES 97.5%, 1 day, on demand for any set of trades or positions: the bank, a legal entity, desk, book, a portfolio, a rule or a basket. |
| **Every input** | **One engine, every input: scenario PVs or path P&L, by trade or by position, any currency, files or Kafka. Source systems keep their own formats.** |
| **How does it work?** | It never reprices. The pricing systems send scenario PVs (in each trade's or position's own currency) or path P&L, at trade or position level, through files or Kafka. Drishti turns each into one internal form, a P&L vector per trade or position, converting PVs with scenario FX into any reporting currency. A scope's vector is the sum of its trades' or positions' vectors, and VaR, ES and every contribution are read from that one vector. |
| **Can I trust it?** | Every answer names its inputs' versions, batch ids, scenario set, method and memory tier; components add up to the VaR exactly; every node gets an official end-of-day record, append-only, kept 7 years. |
| **Does it scale?** | Measured: 1 million rows summed in 58 ms[1] (heap tier); 100,000 × 5 dates on the tiered store, a node in 0.48 ms[2]; 700 limit letters build in the same time[3]. Sized for a large bank (e.g. 2.3M positions, 15,000 portfolios)[4] on the sizing and capacity slides; VaR sharded across a cluster is built. |



[1] Measured: VarScaleBenchmark, 1M trades (VAR_GUIDE §27.2) [2] VarTierBenchmark, 100,000 × 5 dates (§27.3) [3] 10,000 trades (§15.14) [4] derivation A-1

# Why Drishti VaR

01

WHY IT MATTERS

# VaR at a large bank: scale, sources, currencies, intraday

**≈ 2.3M**[1]
positions valued every business date

**≈ 15,000**[1]
portfolios, each a VaR on demand

**5**[2]
reporting currencies built ahead: USD, EUR, CAD, JPY, GBP

**≈ 700**[1]
limit letters: a hierarchy configured by columns

*Illustrative volumes of a large bank (an example, not a client's figures); every other figure in this deck is Drishti's.*

## Scale

- A node's VaR from its float64 aggregate in under a millisecond, whatever its size
- Position grain keeps rows = positions; portfolios are sums of positions

## Pricing systems keep their formats

- Scenario PVs or P&L, by trade or by position, files or Kafka
- Each system its own form; systems mix on one date and may switch day to day

## Multi-currency reporting

- PVs in each trade's or position's own currency, converted with each scenario's FX
- A EUR or CAD VaR is not the USD VaR × today's rate: it carries the currency risk

## Intraday

- Only the changed trades or positions, applied as new − old up every tree
- The official end-of-day record stays the full run's; intraday never restates it

**Status words** *Built things carry no label; only what is not built yet does.*

| PLANNED | NOT BUILT | NOT RUN | PROPOSED | MEASURED | ESTIMATED | ASSUMPTION |
|---|---|---|---|---|---|---|
| not built yet | open, not built | not yet run on AWS | this deck's proposal | on a workstation | scaled from measured | bank to confirm |



[1] Illustrative volumes of a large bank, the product owner's sizing case: ASSUMPTION, for the bank to confirm [2] drishti.var.currencies: the five built ahead by default

# How VaR works

02

# How VaR works in Drishti: add vectors, then read the tail

**1 A P&L vector per trade or position**
509 numbers: its P&L if the market moved as on each scenario day

→

**2 Add the scope's vectors**
a set of trades or positions; overlaps once; longs and shorts net

→

**3 Sort the 509 P&Ls**
worst first; equal P&Ls: the older scenario first

→

**4 Read the tail**
VaR: minus the 6th worst; ES: minus the mean of the 13 worst

## Worked example

ten scenarios, α = 80% so k = 2; one book holding P1 (T1 long, T2 short) and P2 (T3)

| Vector | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| **T1** | 5 | −8 | 3 | −12 | 6 | −2 | 4 | −5 | 7 | 1 |
| **T2** | −2 | 3 | −1 | 4 | −3 | 1 | −2 | 2 | −3 | 0 |
| **P2 = T3** | −4 | 2 | −6 | 1 | −9 | 3 | −1 | 0 | 2 | −7 |
| **P1 = T1 + T2** | 3 | −5 | 2 | −8 | 3 | −1 | 2 | −3 | 4 | 1 |
| **Book = P1 + P2** | −1 | −3 | −4 | **−7** | **−6** | 2 | 1 | −3 | 6 | **−6** |

**VaR**
**6**
2nd worst of the book: −6

**ES**
**6.5**
mean of −7 and −6

**Component VaR**
**1.26 + 4.74**
P1 + P2 = 6: Euler, adds up

**Diversification**
**6**
standalone 5 + 7 − 6

**Incremental, P1**
**−1**
a hedge: removing it raises VaR

## The measures at N = 509

| Measure | Definition |
|---|---|
| **VaR 99%** | minus the 6th worst P&L |
| **ES 97.5%** | minus the mean of the 13 worst |
| **Component VaR** | Euler on ranks 4–8; adds up to VaR |
| **Component ES** | mean over the 13 worst; adds up to ES |
| **Incremental** | VaR(scope) − VaR(scope − part) |
| **Marginal** | component ÷ \|size\| × 1,000,000 |
| **Diversification** | Σ standalone VaR − VaR |
| **Limits** | per node, portfolio; amber 75%, red 90% |

Kupiec test (Paul Kupiec, 1995): is the number of VaR breaches (losses bigger than VaR) consistent with 99%, about 2–3 in 250 days?
Christoffersen test (Peter Christoffersen, 1998): do breaches cluster together, a sign the model reacts too slowly when markets turn?
Basel traffic light: breaches in 250 days at 99%: 0–4 green, 5–9 amber, 10+ red.

CONFIGURABLE

# The window, the confidence, the currencies and the names are settings

## Scenario window

`drishti.var.scenario-window: all | N`

The newest N scenarios of the set (all: 509). Per request, window=250 shows a 1-year view beside the 2-year one: labelled on every answer, never official.

| Window | VaR 99% | ES 97.5% |
|---|---|---|
| **all (509)** | 6th worst | mean of 13 |
| **250** | 3rd worst | mean of 7 |

## Confidence

`DRISHTI_VAR_CONFIDENCE · DRISHTI_VAR_ES_CONFIDENCE`

VaR 99% by default, 95% by configuration; ES 97.5% by default; both also per request. Every label follows the answer (VaR 95% · 1 day). Backtesting stays at 99%.

| VaR at | 509 days | 250 days |
|---|---|---|
| **99% (default)** | 6th worst | 3rd worst |
| **95%** | 26th worst | 13th worst |

**Choosing the window** banks typically use 1 year (≈ 250 days, the regulatory minimum) or 2 years (≈ 500). A shorter window reacts faster to new volatility; a longer one is steadier and keeps more stress days. Compare both with window= before changing the setting.

## Currencies, built ahead

`drishti.var.currencies`

USD, EUR, CAD, JPY, GBP, built in the background after the USD book

`currency-retention-dates`

5: the newest dates kept; older ones built again on first ask

## Names

`DRISHTI_VAR_ROOT_NAME`

the top of every VaR tree (root-names: per hierarchy)

`config/demo/names.yaml`

the demo's bank and legal-entity names

## Inputs and intraday

`drishti.var.inputs`

optional pin per source system: form and grain

`intraday.kafka.window`

5m; early flush at max-trades (500)

# From scenario PVs in their own currency to VaR in any currency

## Inputs

- Pricing systems send each trade's or position's 509 scenario PVs and base PV in its own currency (`scenario-pv`). The USD P&L-vector form stays supported; a date may mix both, each row in one form.
- Scenario FX: the batch's own (`fx-scenario`) when sent, else derived from the FX history. Every cross goes via USD. Horizon: 1 day.
- Converted when the date loads; positions, nodes, captures and official records do not know which form a row came in.

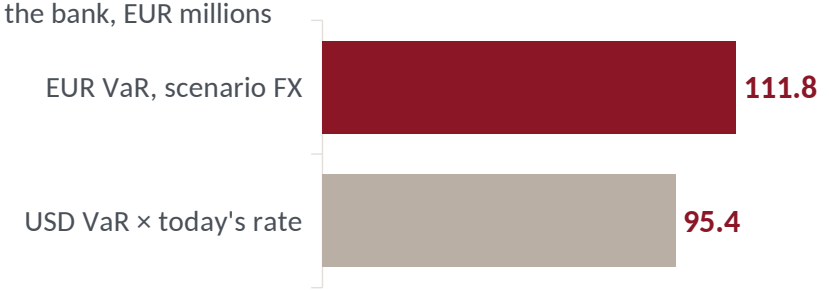
```
USD P&L      PV(s)·FX_c(s) − PV(base)·FX_c(base)
FX derived   FX(s) = FX(base) · H(d_s) / H(d_s − 1)
In r         (p + P&L_USD(s)) / FX_r(s) − p / FX_r(base)
Node in X    Σ_c [PVsum_c(s)·FX_c→X(s) − PVbase_c·FX_c→X(base)]
```

## Reporting currencies

- Built ahead in the background: USD · EUR · CAD · JPY · GBP (`drishti.var.currencies`); the newest 5 dates kept; any other currency with FX on first ask.
- Every node keeps its PVs summed per trade currency: a headline in any currency is exact to the cent.
- Trade pages say the Trade currency (EUR · as priced); official records in USD, others per setting.

## Why EUR VaR ≠ USD VaR × today's rate[1,2]

the bank, EUR millions




The bank on 2026-09-30 holds a net 3.65 bn USD of value; on the scenario days it moves against the euro, a risk a euro-based owner carries and a dollar-based one does not.

| Ccy | Bank VaR 99%[1] | Bank ES 97.5%[1] |
|---|---|---|
| USD | 112,041,646 | 160,143,809 |
| EUR | 111,837,045 | 137,631,546 |
| CAD | 189,801,404 | 230,680,413 |
| JPY | 20,883,226,318 | 23,578,418,999 |
| GBP | 104,139,554 | 125,595,958 |

**On the demo:** the 1,339 Endur trades come as scenario PVs (1,089 USD, 246 EUR, 4 CAD). Bank VaR 112,035,906 → 112,041,646 and ES 160,132,107 → 160,143,809: the FX move of the 250 EUR and CAD trades. A USD trade reproduces its vector exactly.



[1] Computed by the engine on the committed synthetic demo data, 2026-09-30 (VAR_GUIDE §15.9–15.11) [2] 95.4 = the USD VaR at today's EUR rate, as the guide quotes it

# Intraday: only what changed, applied as new – old up the tree

**Morning full run**
RUN_KIND FULL: every live trade or position; the date loads; official records taken from it

→

**Changed rows**
Trades or positions amended, new, cancelled; same scenario set; own BATCH_ID, later COMPUTED_AT

JSONL files

→

**Lake: appended batch**
Files: drishti.py var intraday --load (lake build --mode append). Kafka: one batch per window

Kafka micro-batches

→

**Apply: new – old**
Into the book, every node above (every hierarchy), the currency totals and built currencies

→

**Answers say intraday**
Badge in the status line, intraday on every answer; morning hot file never changed

| Case | What the engine does |
|---|---|
| **Same trade or position, several batches** | latest COMPUTED_AT wins across files and Kafka; an equal one keeps the first |
| **COMPUTED_AT before the morning run** | ignored and counted |
| **Cancel (CANCELLED, vector null)** | removed from every scope |
| **New trade** | book from TT_BOOK, position by the position rules |
| **Moved to another book or position** | old ordinal cancelled, added in its new place |
| **Another scenario set** | row refused (DRS-5203); the date stays as it is |
| **A full run of the date** | date reloads; the intraday layer is dropped |

*Cost of a batch ∝ rows changed × nodes above × currencies built, never the size of the book.*

### Kafka, as built

- A message is JSONL text: one or more records, PVs or P&L, a cancel as a marked line
- A window every 5 min (drishti.var.intraday.kafka.window); early flush at 500 trades; admin “Apply now”
- Each window written to the lake as ONE intraday batch (on Iceberg, one append), then applied like a file
- Replaying the topic after a restart writes nothing twice

**The guarantee  incremental = the same data loaded as one run, to the cent, on every node**

Measured[1]: 200 batches of 3 trades on one date; a restart equal too; batch time does not grow. Official numbers come only from full runs.



[1] VarIntradayApiTest: 200 batches of 3 trades, equal to the cent to the same data loaded as one run, on every node (VAR_GUIDE §8.4)

# One explorer for any scope, and the same answers everywhere else




*VAR DESK DESK-RATES IN EUR: the scope tree and tiles in EUR, the basket tray, the P&L path, and How this number was built naming the reporting currency, the input form, the scenario FX and the crosses via USD (2026-09-30).*

**9–14 ms**[1] click to painted, a desk, answers fetched ahead (target 300 ms)

**390 px** on a phone the three columns stack, no sideways scroll

- **Scope tree and six tiles** MGMT, LEGAL or the limit letters (a tree by columns) with VaR, ES and limit bars; VaR, ES, sum of standalone, diversification, limit use, backtest exceptions.
- **Five tabs** P&L path (509 days, worst 25, who lost on a day) · VaR/ES history and backtest · contributions · captured results · In other currencies.
- **Reporting currency picker** USD, EUR, CAD, JPY, GBP and More…; every panel follows; kept in the link; VAR … IN EUR.
- **How this number was built** the scope as resolved, its trades and positions, hidden and unknown ids, batch, data versions, method, tier, timing.
- **Baskets and pre-deal what-if** any mix of portfolios, positions, trades, counterparties and rules, live; a booked trade's vector × a factor through the Python engine.
- **Captures, official records** immutable captures, compare with now; each node's official history.
- **Everywhere else** trade (Trade currency), position, portfolio, desk and LE pages; drishti.py var … --ccy; /api/v1/var; VRES and VWIF; the BI VaR report; Home facts.
- **Governed** root name drishti.var.root-name; per-node positions or aggregate access; masks; every answer audited.

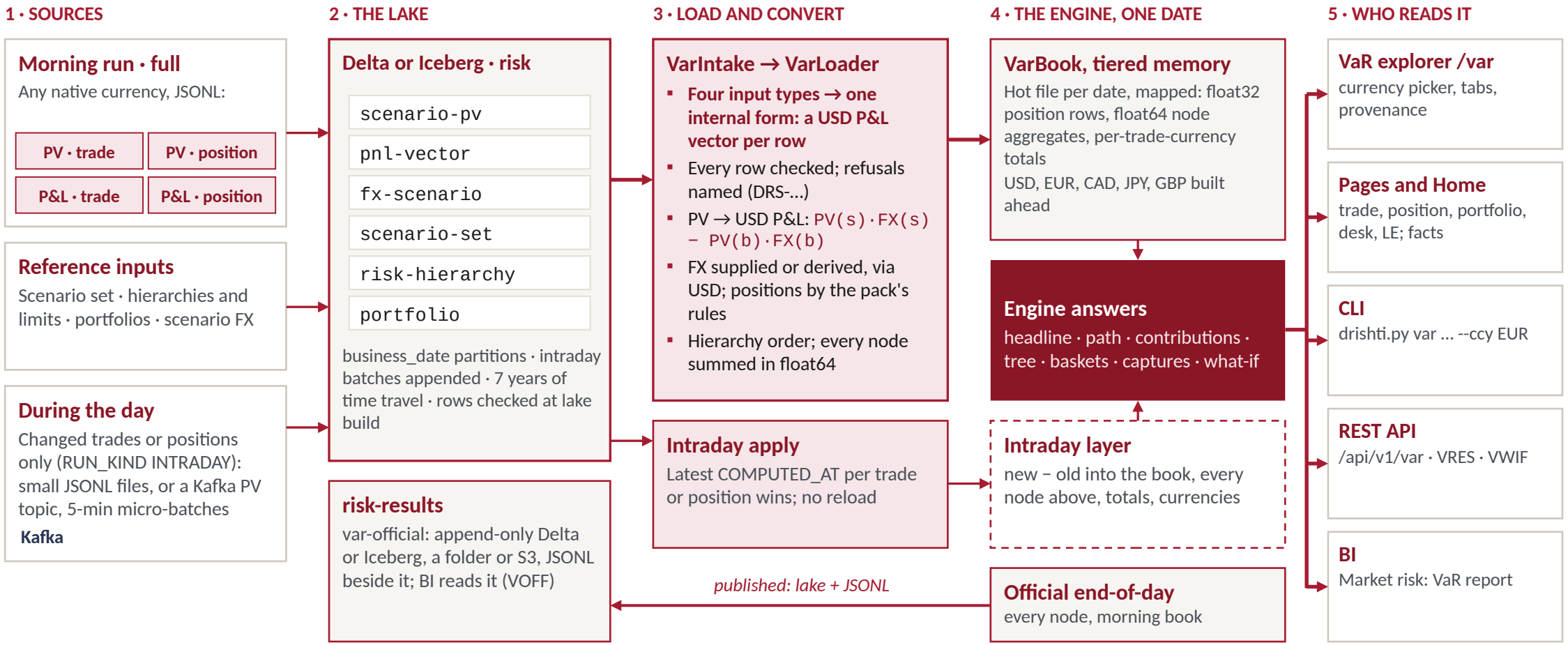


[1] Measured: painted in 9–14 ms with answers fetched ahead, 30–35 ms without (VAR_GUIDE §27.2); the 300 ms target is the product owner's

# Data in

03

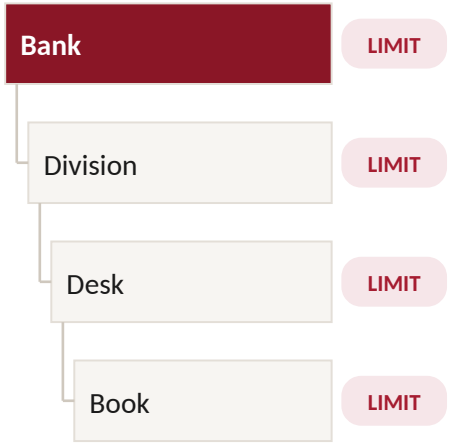
# From the pricing systems to every screen, in one direction



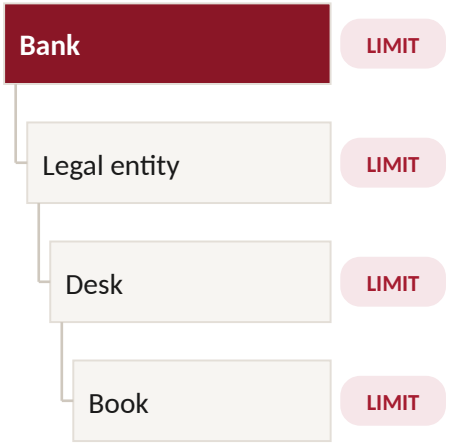

*Each batch's data-load signal* `POST /api/v1/packs/market-risk/loads` *tells the engine: a full run reloads the date; an intraday batch is applied on top. The lake is the record: a restart rebuilds everything, the intraday layer included.*

# Three trees: management, legal entity and the limit letters

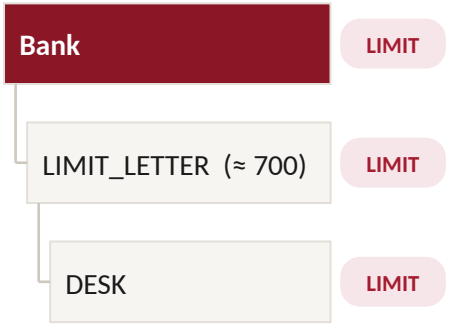
### MGMT · management




### LEGAL · legal entity




### LL · limit letters




## Configured by columns

- Levels are columns every row carries (LIMIT_LETTER > DESK); nodes are the distinct paths of the values
- A letter may hold part of a book; a position under two letters is split (<position>@<letter>) and counted
- A source without the column skips that hierarchy alone, with a DRS-5211 warning; the date, MGMT, LEGAL and exposure answer as before
- Official records for every hierarchy; exposure builds the same trees

```
# the market-risk pack's hierarchies: block, as built
hierarchies:
  MGMT:  { title: Management, source: risk-hierarchy }
  LEGAL: { title: Legal entities, source: risk-hierarchy }
  LL:    { title: Limit letters, levels: "LIMIT_LETTER > DESK", word: LL, limits: var-limit }
```

## Limits and the picker

- var-limit: HIERARCHY, NODE_ID, LIMIT_VAR (USD); no row: no limit, never zero
- Limit bar amber 75%, red 90%; LIMIT and LIMIT_USE in official records
- The explorer's hierarchy picker lists the date's trees (hierarchy=LL kept in the link)

MEASURED **700 letters build in the same time as none, and the VaR of all 1,412 nodes takes about 30 ms** (10,000 trades; build 65–99 ms against 53–197 ms without). Scope word LL:LL-0003; node ids are paths, LL~LL-0003/DESK-RATES.

ANY INPUT CONTRACT

# One engine, every input: source systems keep their own formats

| | Scenario PVs<br>native currency | Path P&L vectors<br>any currency |
|---|---|---|
| **By trade** | ✓ in the trade's currency | ✓ a currency a row |
| **By position** | ✓ in the position's currency | ✓ a currency a row |

## Delivery routes

**Files, JSONL**
the morning run; small intraday batches appended

**Kafka, micro-batched**
JSONL messages; 5-min window, early flush, Apply now

**Intraday, either route**
changed trades or positions, as new − old; latest wins

```
{"VALUE_TYPE":"PV",  "GRAIN":"TRADE",    "TRADE_ID":"1000279", "CURRENCY":"EUR", "BASE_PV":…, "PVS":[…509…], "COMPUTED_AT":"…", …}
{"VALUE_TYPE":"PV",  "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"JPY", "BOOK":"…", "BASE_PV":…, "PVS":[…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"TRADE",    "TRADE_ID":"MX-…",    "CURRENCY":"USD", "PNL":[…509…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"USD", "PNL":[…], …}
```

## Mix and switch

- Each system sends its own form and grain; systems mix on one date.
- A system may switch day to day; each date loads as it stands.
- Old files: `drishti.py convert --set …`

```
# optional cross-check
drishti.var.inputs:
  ENDUR:   {form: pv,  grain: trade}
  CALYPSO: {form: pnl, grain: trade}
  default: {form: any, grain: any}
```

## Guarantees

- One form and grain per system per date (DRS-5210): no double counting
- Fields missing or wrong (DRS-5208), or against the pin (DRS-5209): refused, never guessed
- Provenance shows what each system sent

**Caveat** a P&L vector sent without its PV converts to another reporting currency only approximately (at each scenario's rate, without the FX move of its base value; counted as `tradesWithoutPv`). Scenario PVs give exact multi-currency VaR.

# Six input contracts, JSONL on the wire, every check named

**JSONL:** one JSON object a line, UTF-8, .jsonl or .jsonl.gz; each line exactly its contract's fields. Key fields of the as-built contracts:

| `scenario-pv` SPV · a trade × date × set | |
|---|---|
| `TRADE_ID … TT_BOOK` | trade; LE, desk, book, letter |
| `CURRENCY · BASE_PV` | PVs' currency; value today |
| `SCENARIO_PV [509]` | per scenario; null = failed |
| `RUN_KIND` | FULL or INTRADAY |
| `SCENARIO_SET_ID` | and BATCH_ID: the run |

| `pnl-vector` PNLV · a trade × date × set | |
|---|---|
| `VECTOR_ID · TRADE_ID` | PNL-<trade>-HS-509 |
| `PNL [509]` | USD P&L, gain positive |
| `PV · REPORTING_CCY` | USD value today; USD |
| `STATUS` | ok, proxied or failed |
| `RUN_KIND` | as scenario-pv |

| `fx-scenario` FXS · a currency × date | |
|---|---|
| `FX_SCENARIO_ID` | FXS-<CCY>-HS-509 |
| `CURRENCY` | AUD, CAD, EUR, JPY … |
| `RATE` | today, USD per one unit |
| `RATES [509]` | per scenario, the set's order |
| `not sent?` | derived from the FX history |

| `scenario-set` SSET · one row a date | |
|---|---|
| `SCENARIO_SET_ID` | HS-509 |
| `SCENARIO_DATES` | 509 days, oldest first |
| `WINDOW_START … END` | 2024-09-17 … 2026-09-29 |
| `HORIZON_DAYS` | 1 |
| `REVALUATION` | full or sensitivities |

| `risk-hierarchy` HIER · a node × date | |
|---|---|
| `NODE_ID` | MGMT~<id> or LEGAL~<id> |
| `HIERARCHY · LEVEL` | MGMT, LEGAL…; open values |
| `PARENT_ID · NAME` | the tree; display name |
| `BOOKS` | every book beneath |
| `limits` | in var-limit (VLIM), USD |

| `portfolio` PORT · a portfolio × date | |
|---|---|
| `PORTFOLIO_ID · NAME` | PF-…; scope PORT:<id> |
| `TYPE` | HIERARCHY, RULE or BASKET |
| `DEFINITION` | NODE, RULE or MEMBERS |
| `OWNER` | a user or a team |
| `limit` | var-limit, hierarchy PORTFOLIO |

**Required on every record:** `VALUE_TYPE`, `GRAIN` with its id, `CURRENCY`, `SOURCE_SYSTEM`; GRAIN matches the id. Old files: `convert --set`

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **DRS-5201** length, null, NaN | **DRS-5206** a PV failed | **DRS-5207** no scenario FX | **DRS-5208** record fields wrong | **DRS-5209** against the pin | **DRS-5210** 2nd form or grain | **DRS-5202** trade twice | **DRS-5203** two runs |

# Computed, never stored

04

# Computed on demand from trade-grain inputs, never stored

**The principle.** A total is computed from the trades it adds up, once per business date and data version, and shared by everyone who may open the page. A stored copy is shown beside it with the difference, until it is retired.

| Computed | What it is |
|---|---|
| `NSMTM` | a netting set's MtM from its live trades: net, gross parts, netting benefit, CE, past maturity |
| `CPMTM` | a counterparty's MtM and its netting sets, from the trade valuations |
| `SSPNL` | a stress scenario's P&L: every desk, its limit use, breaches |
| `Checks panels` | collateral after haircut, credit-limit headroom, the SA-CCR EAD identity, stress limit use |
| `Exposure engine` | members and MtM from trade valuations; collateral after haircut; limit from credit-limit |

**Live** is one rule everywhere: a live status and a maturity not before the valuation date. A live trade past maturity is in no total, and is listed as a data-quality warning.




*A netting set: computed, stored and the difference*




*Checks: the collateral balance and the credit limit*

## Why

- One definition per figure: no second copy to drift and no batch job to forget
- Every total traceable to the trades it adds up; a stored copy is checked, not trusted

# Reconciled to the stored figures, retired fields guarded, VaR unchanged

**55 of 76**[1]
netting sets: derived net MtM equals the stored one

**21 explained**[1]
827 live trades past maturity (MtM 16.7m); added back, 76 of 76 agree to the dollar

**112,041,646**[1]
VAR BANK, USD, 2026-09-30: unchanged, no VaR input moved

## Exposure on its real inputs[1]

collateral after haircut, 2026-09-30, USD

| Scope · figure | Stored inputs | Computed inputs |
|---|---|---|
| BANK · peak PFE 95% | 2,858,060,180 | 2,897,620,439 |
| BANK · EPE | 945,577,981 | 947,202,991 |
| BANK · CE | 1,532,007,701 | 1,586,811,699 |
| CP-ALDERSHOT · CE | 217,565,006 | 220,250,163 |
| NS-ALDERSHOT-NY · CE | 212,580,686 | 214,706,654 |

Holdings 6,304.4m at market value, 6,229.6m after haircut: less held raises a collateralised set's exposure, less posted lowers it.

## Retired fields, guarded

- `netting-set.UTILISATION`, `risk-hierarchy.LIMIT_VAR`, `portfolio.LIMIT_VAR`
- The build fails if one returns (tools/contracts.policy.yaml, removed:); VaR limits live only in var-limit

## The difference is shown, not hidden

- The netting set's panel: computed, stored, difference
- CP-ALDERSHOT: stored 520,405,666, derived 518,314,005 (18 trades past maturity)

NOT BUILT

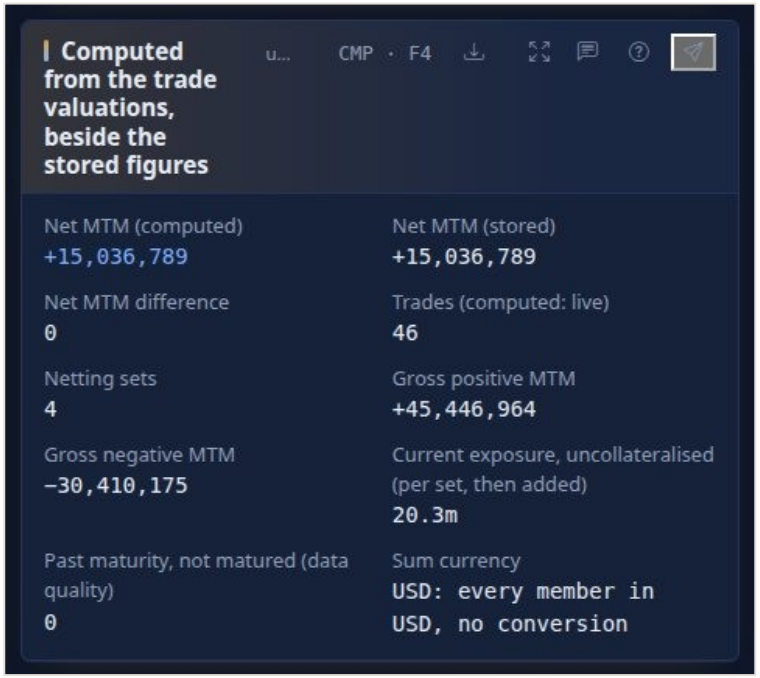
## Still open

retiring the stored netting-set and counterparty MtM with the netting-set engine; currency conversion on derived totals (all USD today); BI's MtM measures filtered to live trades; SA-CCR's stored RC still before haircut.

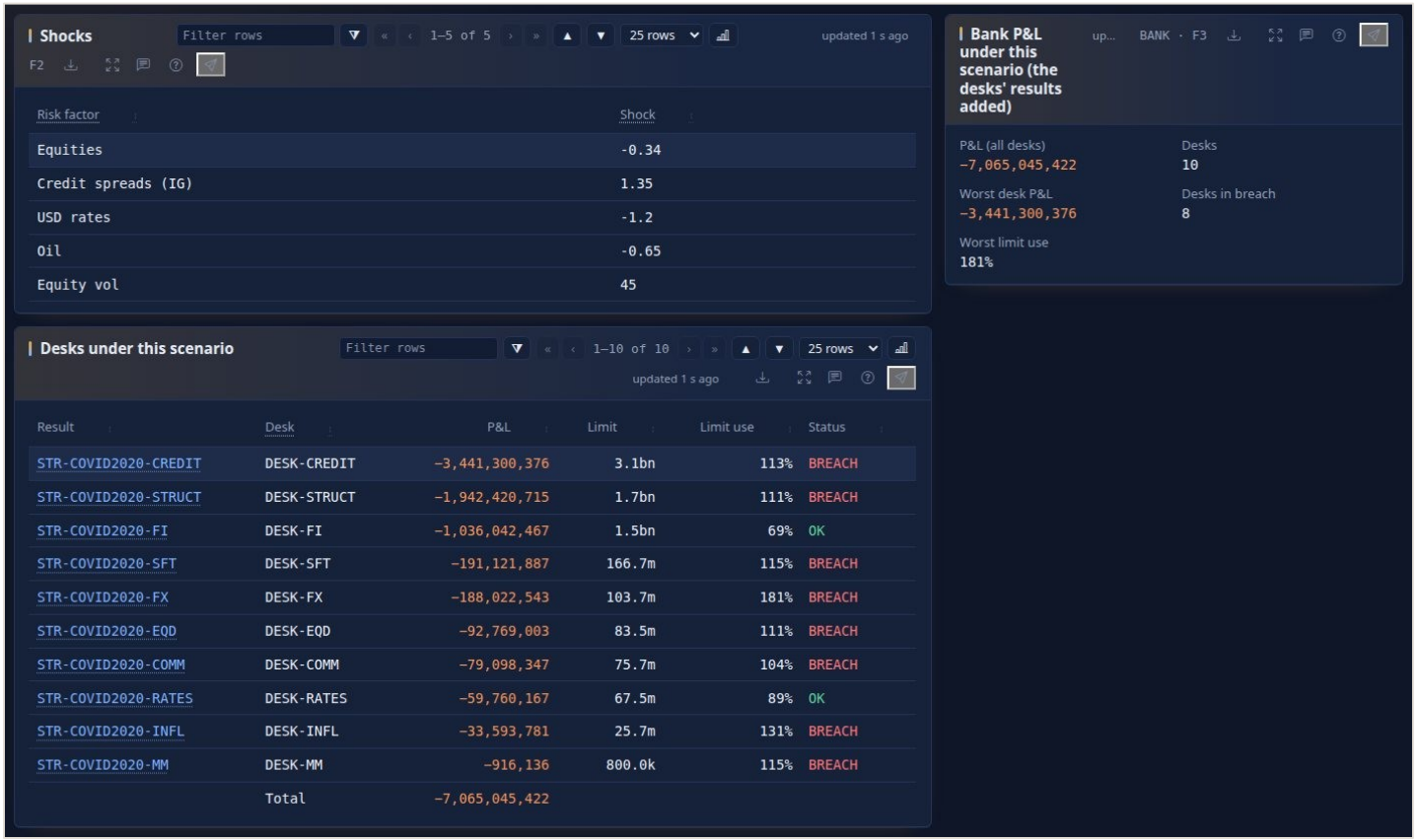


[1] Measured on the committed test data, 2026-09-30 (COMPUTED_ATTRIBUTES.md §13.2–13.4)

# On the pages: the counterparty and a stress scenario, computed




*A counterparty, computed from the trade valuations*




*A stress scenario: the bank's P&L and every desk under it (SSPNL)*

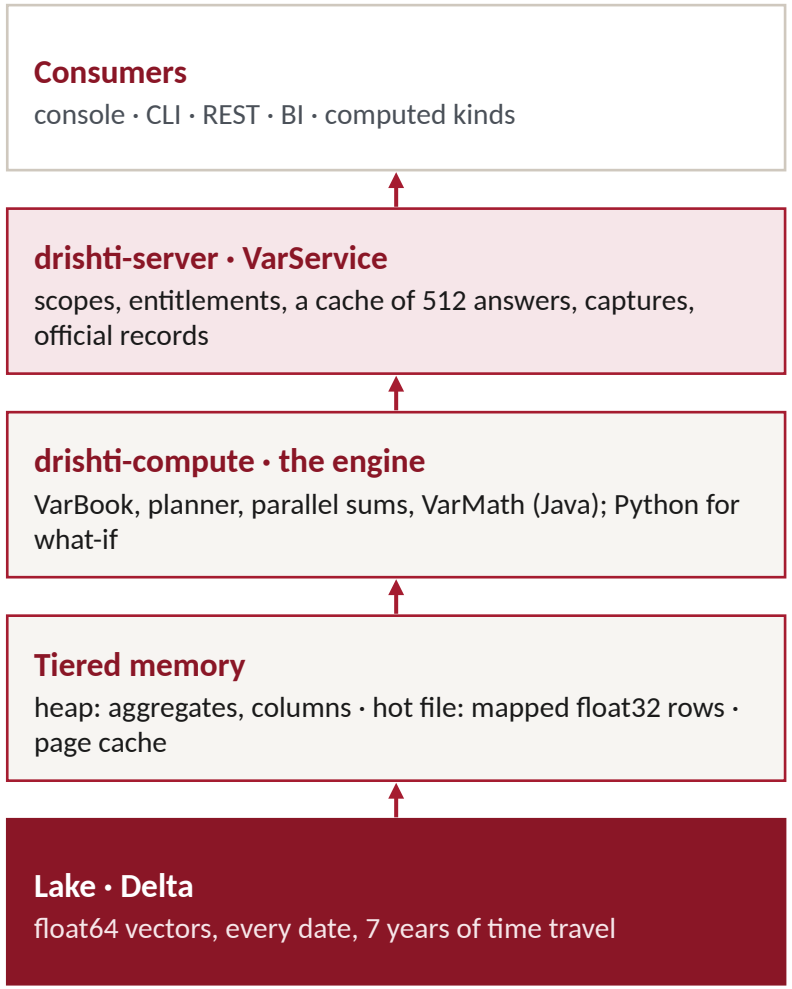
**Read from the trades, not copied.** A page shows the computed figure with its source, its sum currency and its checks; a stored copy, while it exists, sits beside it with the difference.

# Scale and capacity

05



# Why it is fast: vectors add, one date stays hot, sums run on every core




**Vector math[1]**
509 doubles = 4,072 B a trade or position; hot rows float32 (2,036 B); sums in float64

**Memory-mapped hot file**
one file a date; only touched pages resident; 2 dates open, 60 kept on disk

**Planner over aggregates**
a node is its aggregate; Markets minus three trades = 4 vector operations

**Parallel sums**
slices on every core, added in a fixed order: the same bits every time

**Currencies built ahead[2]**
five books in the background; 21.4 MB a currency a date on the demo

**Per-trade-currency totals**
a headline in any currency from a node's few currency sums, exact

**Intraday increments**
cost ∝ rows changed; Kafka: one commit and one apply per window

**Cluster and history**
Drishti*net replicas; VaR sharded across servers; lake history

| Measured[3] | Time | On |
|---|---|---|
| A node's VaR and ES | 0.48 ms | tiered store, 100,000 trades |
| The whole bank from float32 rows | 3.4 ms (≈ 60 GB/s) | tiered store, 100,000 trades |
| The whole bank from float64 rows | 58 ms (≈ 70 GB/s) | heap tier, 1,000,000 trades |
| A basket of 50,000 scattered positions | 2.6 ms + ≈ 45 ms ids | tiered store, 100,000 trades |
| First load of a date (after the lake read) | 0.8–1.2 s | tiered store, 100,000 trades |
| Loading the demo date | 572 ms | 9,572 trades, the server |



[1–3] How these figures were derived: A-1, in the appendix (How it scales)

# Bank scale, from first principles and the measured figures

| Per business date | Arithmetic | Result | Basis |
|---|---|---|---|
| **Vectors, float64 (the lake)[1]** | 2.3M × 509 × 8 B | 9.37 GB | arithmetic |
| **Vectors, float32 (hot rows)[1]** | 2.3M × 509 × 4 B | 4.68 GB | arithmetic |
| **Hot file, USD book[2]** | 237.7 MB per 100k × 23 | ≈ 5.5 GB | *estimated* |
| **Heap, one open date[2]** | ≈ 80 MB per 100k × 23 | ≈ 1.8 GB | *estimated* |
| **First load, after the lake read[2]** | 0.8–1.2 s per 100k × 23 | ≈ 18–28 s | *estimated* |
| **Reopen a date from its file[2]** | 23 ms per 100k × 23 | ≈ 0.5 s | *estimated* |
| **Whole bank from rows, resident[3]** | 4.68 GB at ≈ 60 GB/s | ≈ 80 ms | *estimated* |
| **... cold, from NVMe[3]** | 4.68 GB at ≈ 3 GB/s | ≈ 1.6 s | *estimated* |
| **A node, any size[3]** | its float64 aggregate | ≈ 0.5 ms | **measured** |
| **4 extra currencies, disk[4]** | 250 MB × 4 per 100k × 23 | ≈ 23 GB a date | *estimated* |
| **... 5 retained dates; build[4]** | 23 GB × 5; 4.3 s per 100k × 23 | ≈ 115 GB; ≈ 99 s | *estimated* |
| **USD hot files, 60 dates[5]** | 5.5 GB × 60 | ≈ 330 GB disk | *estimated* |
| **Lake, a year (Parquet ≈ 0.75×)[6]** | 9.37 GB × 0.75 × 252 | ≈ 1.8 TB; ≈ 12 TB in 7 y | *estimated* |

**Position grain keeps rows = positions: no trade matrix in the hot file.** *Estimated = scaled linearly (× 23) from the measured 100,000-trade figures (VAR_GUIDE 27.3, 15.11); not measured at 2.3M.*

## 15,000 portfolios[7]

- A portfolio is the sum of its positions: the cost is all portfolios' positions together × 509 additions.
- Assumed 130 positions each: 1.95M rows, 3.97 GB of float32 → ≈ 0.07 s at 60 GB/s, ≈ 0.8 s scattered; per currency, estimated.
- Roll-ups: a HIERARCHY portfolio is a node aggregate; whole books cost one add each; a node minus a few rows is aggregate − rows (built).
- Store them: 15k × 509 × 8 B ≈ 61 MB a date a currency; float32 halves it (proposed).

## Totals and intraday[8]

- Currency totals: 510 × 8 B per node per trade currency; (5,000 MGMT and LEGAL + ≈ 700 limit-letter nodes) × 10 ≈ 233 MB (assumed shape).
- Per changed trade or position: ≈ 11 node vectors (three trees) × 5 currency books ≈ 55 additions of 509: microseconds; the lake append dominates (≈ 1 s a batch).
- A headline in any currency: a node's few trade-currency sums, converted: microseconds.

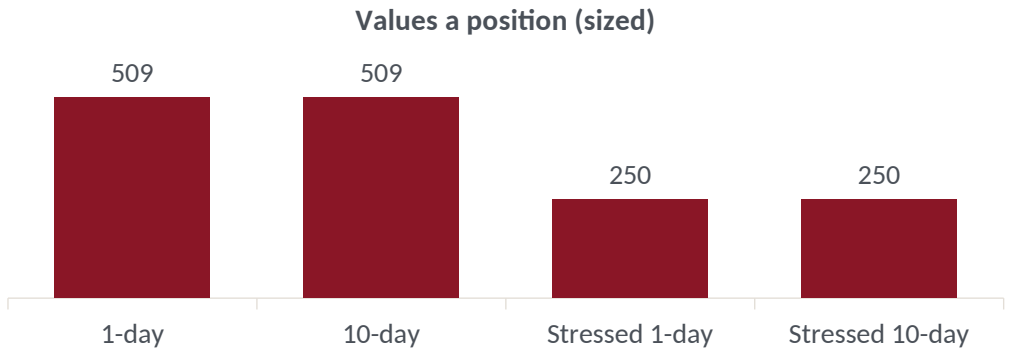


[1–8] How these figures were derived: A-1, in the appendix (Sizing)

# 10-day and stressed VaR: the added workload

PLANNED

| Measure | Vector a position | × 1-day |
|---|---|---|
| 1-day VaR and ES | 509 scenarios (2 years) | 1.00 |
| 10-day (b), overlapping | 500 (509 − 9), sized 509 | + 1.00 |
| Stressed 1-day VaR, ES | ≈ 250 (12 months, e.g. 2008–09) | + 0.49 |
| Stressed 10-day | ≈ 250 overlapping | + 0.49 |
| Total, sized[1] | 1,518 values a position | 2.98 |




| Per date and server | 1-day only | All four | Label |
|---|---|---|---|
| Vectors, float64[2] | 9.37 GB | 27.9 GB | ESTIMATED |
| Rows, float32[2] | 4.68 GB | 14.0 GB | ESTIMATED |
| USD hot file[3] | 5.47 GB | 16.3 GB | ESTIMATED |
| Heap, an open date[4] | 1.84 GB | 2.23 GB | ESTIMATED |
| First open: lake read[5] | 18–70 s | 54–209 s | ESTIMATED |
| First open: build[5] | 18–28 s | 55–82 s | ESTIMATED |
| Other currency, asked[6] | 5.75 GB | + 11.4 GB | ASSUMPTION |
| Local NVMe a server[7] | 458 GB | 1,183 GB | ESTIMATED |
| RAM a server[7] | 76 GB | 151 GB | ESTIMATED |
| Lake, 7 years[7] | 16.1 TB | 48.2 TB | ESTIMATED |

**Method (ASSUMPTION):** 10-day by (b), overlapping returns, the conservative case; (a) √10 × 1-day adds no data, × 1.49 only.
**Currencies (ASSUMPTION):** USD builds all four sets with the date; the others build 1-day ahead and the rest on first ask.

**About 3× the 1-day footprint per date.** With (a) √10 instead[8]: × 1.49, vectors 18.5 TB over 7 years. Sums per measure are unchanged: a node 0.5 ms, the whole bank ≈ 80 ms for a 509-wide set, ≈ 40 ms for a stressed one (estimated).



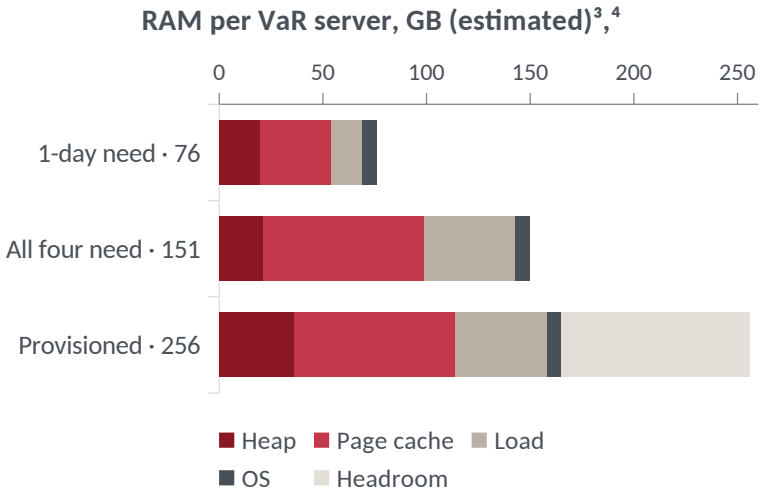
[1–8] How these figures were derived: A-2, in the appendix (10-day and stressed VaR)

# Capacity: processes and memory, 1-day, 10-day and stressed

## Processes, recommended one server holds each date's whole VaR

| Role | No. | Holds and does |
|---|---|---|
| Official EOD server | 1 | captures every node of every tree and measure; publishes (DRISHTI_VAR_OFFICIAL here only) |
| Intraday writer | 1 | Kafka PV topic, one lake batch a 5-minute window, and files (DRISHTI_VAR_PV_STREAM here only) |
| UI/API VaR replicas | 2 | each maps today and yesterday whole, applies intraday; Drishti*net failover |
| Console hosts | 2 | 4 workers each; load balancer, sticky sessions; one session secret |
| Cluster, sharding | – | servers as one; a date's VaR split by shard (next slides) |

| Per VaR server | Need | Provisioned | Large (2×)[8] |
|---|---|---|---|
| vCPU[6] | 32 | 64 | 96 |
| -Xmx[1] | 21 GB | 36g | 60g |
| Memory budget, MB[2] | 14,000 | 23,000 | 46,000 |
| RAM[4] | 151 GB | 256 GB | 512 GB |
| Local NVMe (tier.dir)[5] | 1.18 TB | 3.8 TB | 5.7 TB |




### CPU: 32 vCPU needed → 64[6]

- Sums are bandwidth-bound: 24 threads gave ≈ 60 GB/s (MEASURED); all four sets of the bank ≈ 233 ms
- + 8 vCPU for a load, currency builds, intraday, requests; 32 ÷ 0.61 = 52 → 64

**39% headroom:** the projected peak uses at most 61% of what is provisioned: need ÷ 0.61 (× 1.64). Set DRISHTI_VAR_MEMORY_MB: the default ¼ of the heap (9 GB at 36g) refuses a 14 GB date.

**Consoles[7]:** 2 hosts × 4 workers; 40 pages/s a worker (MEASURED); 50 users at once (ASSUMPTION): peak 25 pages/s ÷ 0.61 = 41 against one host's 160.

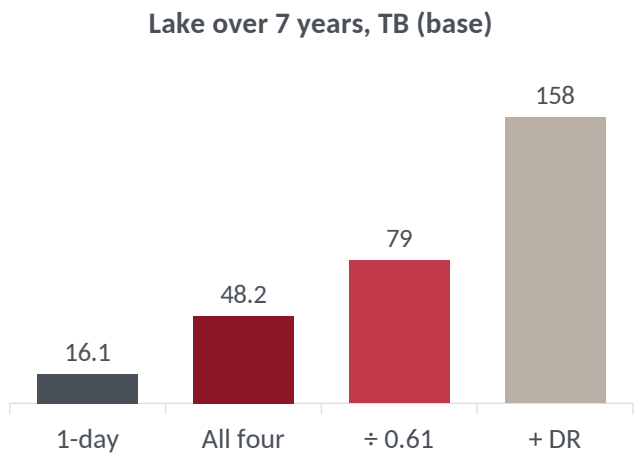


[1–8] How these figures were derived: A-2, in the appendix (Processes and memory)

# Capacity: storage over 7 years

| Item | Arithmetic (all four sets) | 1-day | All four | Label |
|---|---|---|---|---|
| P&L vectors, float64[1] | 27.9 GB × 0.75 × 1,764 dates | 12.4 TB | 37.0 TB | ESTIMATED |
| Row columns (terms)[2] | 2.3M × ≈ 100 B × 1,764 × 4 sets | 0.4 TB | 1.6 TB | ASSUMPTION |
| Scenario PVs as well[2] | the vectors again, if both kept | + 12.4 | + 37.0 | ASSUMPTION |
| Trades, if held[3] | 1.5 KB × 2.3M × 1,764 | 6.1 TB | 6.1 TB | ESTIMATED |
| Positions (membership)[3] | 41 B × 2.3M × 1,764 | 0.17 TB | 0.17 TB | ESTIMATED |
| Official results[4] | ≈ 29 KB × 5,700 nodes × 1,764 × 2.98 | 0.29 TB | 0.88 TB | ESTIMATED |
| Intraday batches[5] | 230k × 9.1 KB × 1,764 × 2 | 2.5 TB | 7.4 TB | ASSUMPTION |
| Restatements[5] | 2% of dates × 20.9 GB × 1,764 | 0.25 TB | 0.74 TB | ASSUMPTION |
| Delta log, checkpoints[6] | daily 0.14 × 2.98 (often: 4.9) | 0.14 TB | 0.42 TB | ESTIMATED |
| Need, base[7] | 37.0 + 1.6 + 7.4 + 0.9 + 0.2 + 0.7 + 0.4 | 16.1 TB | 48.2 TB | ESTIMATED |
| Provisioned (÷ 0.61)[7] | 39% headroom; DR doubles it | 26.4 TB | 79 TB | ESTIMATED |
| Need, upper[7] | + PVs, trades, frequent checkpoints | 36.1 TB | 95.7 TB | ESTIMATED |

*With a DR copy: 158 TB provisioned (base), 314 TB (upper). Delta or Iceberg, not both. 7 × 252 = 1,764 dates; Parquet ≈ 0.75 × raw; × 2.98 = all four sets.*




## Local NVMe, per server[9]

- USD 16.3 GB × 60 dates 978 + currencies 161 + a load 44 = 1,183 GB ÷ 0.61 → 3.8 TB; rebuilt from the lake, never on EFS

## The 250-day window[8]

- 1-day and 10-day cut to 250: 1,518 → 1,000 values (× 0.66); float32 14.0 → 9.2 GB; NVMe ≈ 780 GB
- The lake shrinks (37.0 → 24.3 TB) only if the bank sends 250-day sets



[1–9] How these figures were derived: A-3, in the appendix (Storage over 7 years)

# Capacity for a sharded cluster: 6 owners, 2 coordinators, 2 consoles

**The scenario** a bank of 2.3 million positions and 15,000 portfolios, with 1-day, 10-day and stressed VaR: 1,518 values a position, the conservative case of the single-server slides.

| Role | Count | Heap (-Xmx) | RAM a server | vCPU | Local NVMe | Label |
|---|---|---|---|---|---|---|
| **VaR shard owners** | 6 | 36g[2] | 128 GB (need 69)[3] | 16[3] | ~650 GB (need 394)[1] | ESTIMATED |
| **Coordinators (coordinator-only, no shards)**[4] | 2, failover | 16–20 GB | 32 GB | 8 | ~50 GB | ASSUMPTION |
| **Consoles**[4] | 2, load balancer | – | 8 GB | 4 | – | ASSUMPTION |

```
# the market-risk pack's sharding: block
sharding:
  enabled: true
  key: [LEGAL_ENTITY, DESK]
  replicas: 2       # two zones
  target-shard-mb: 2048
# coordinators: coordinator-only: true
# Java 25 -XX:+UseCompactObjectHeaders
```

### Each shard on two servers

- Shards are legal entity and desk ranges near 2 GB; a desk is never split
- Any one server can fail: the other replica answers
- Replacing a server moves about 1/6 of the rows

### Smaller options

- **1-day VaR only**[5]**:** 4 owners, 24g heap, 96 GB RAM (need 47), plus 2 coordinators
- **Minimum start**[6]**:** 3 owners and 1 coordinator, 36g heap; 192 GB RAM with 2 replicas (need 110), 128 GB with 1 (less failure headroom)

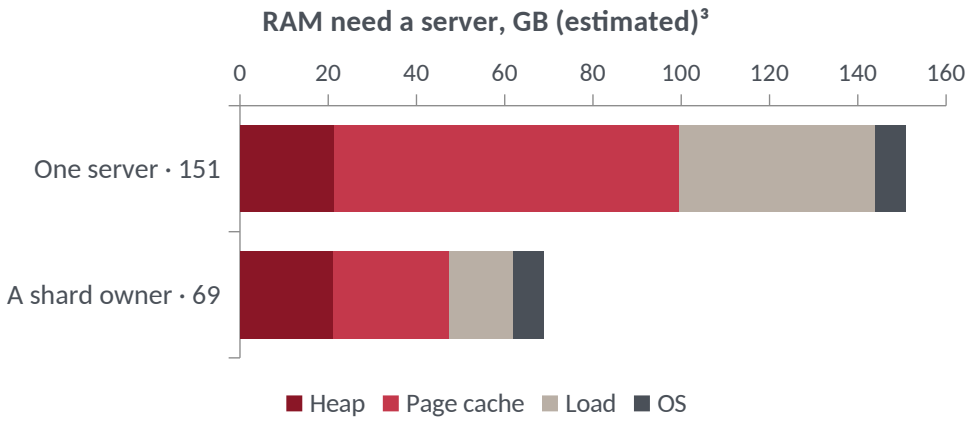
PLANNED **10-day and stressed VaR are not built yet;** they are sized here because they triple the footprint. VaR sharding itself is built (off by default). 1-day only, six owners would need about 39 GB each.[5]

**Estimated, not measured at 2.3M.** Every figure is scaled × 23 from the measured 100,000-trade benchmark. Next: the 200K benchmark, then a 2.3M load on one real bank server before ordering hardware.



[1–6] How these figures were derived: A-3, in the appendix (A sharded cluster)

# Sharding divides the cache and the disk, not the heap




| One server against six owners | One server | An owner |
|---|---|---|
| **RAM need** | 151 GB | 69 GB |
| **vCPU** | 32 needed | 16 (96 in all) |
| **Local NVMe need** | 1.18 TB | 394 GB |

ESTIMATED the nightly official run over 15,000 portfolios, spread over 6 owners: minutes[7]

## Divided by the shards

- Page cache for the hot files: 78.4 GB × 2 replicas ÷ 6 owners = 26.1 GB[1]
- A load's spill files: 44.2 × 1/3 = 14.7 GB; NVMe 1,183 × 1/3 = 394 GB

## Not divided: the heap

- Every member keeps the date's skeleton for all 2.3M positions (ids, columns, book and node aggregates), so it can answer any scope: about 2 GB an open date
- Two dates, the answer cache, × 1.5: 21.2 GB need → -Xmx36g with the 39% headroom; 24g for 1-day only[2],[5]

## An owner: 21 + 26 + 15 + 7 = 69 GB[3]

- ÷ 0.61 = 113 → 128 GB machines (96 GB for 1-day only)
- 15,000 portfolios are light: memberships about 31 MB[7]; each portfolio a sum of its rows, in parallel across shards

**Failure and growth.** With 2 replicas any one server can fail and every scope still answers; replacing one moves about 1/6 of the rows. More positions add owners, not bigger machines. All figures ESTIMATED (× 23 from the measured 100K benchmark); next the 200K benchmark, then a 2.3M load on one real server.



[1–7] How these figures were derived: A-3, in the appendix (A sharded cluster)

# How many people at once: what risk users can expect

**300–500**[5] ESTIMATED

people active at once (1,500–5,000 named)

**330–650 req/s**[2] ESTIMATED

sustained to peak, the 6 + 2 + 2 cluster

**The console** ESTIMATED

is the first limit, not the VaR servers

**Measured**[1] one 24-core server, 8 GB heap, a 1.16M-trade lake

| What | Result |
|---|---|
| **Mixed reads, 100 users** | ≈ 140 req/s, 0 failures |
| **Single trade p95, 10 / 50 users** | 0.08 s / ≈ 1 s; then queues |
| **One console, 50 users** | ≈ 69 req/s; search ≈ 4 s (open) |
| **VaR of a node, one user** | under 1 ms |
| **Bank minus a desk · 50K-position scope** | 2.6 ms · 52 ms |
| **Whole-bank filter, 1M trades** | 58 ms |

*Mix: 80% trades, 10% counterparty-trade pages, 5% netting sets, 5% searches.*

**Estimated for the cluster** 8 servers, each answers any scope

| Layer | Capacity | Basis |
|---|---|---|
| **Servers**[2] | ~650 req/s peak, ~330 sustained | 140 measured on 24 cores → ~93 a 16-vCPU, ~47 an 8-vCPU server |
| **VaR queries**[3] | thousands/s for nodes; ~10–25/s whole-bank filters | nodes from totals; a filter reads ~4.7 GB over 6 owners |
| **Consoles (2 × 4 vCPU)**[4] | ~100–140 page loads/s | 69 measured a console |
| **Active users**[5] | ~300–500 | consoles first: a page every 10 s, 3–5 calls: 0.3–0.5 req/s each |
| **Named users**[6] | ~1,500–5,000 | 10–20% active at once |

**What limits it first** 1 the consoles: add more behind the load balancer (sticky sessions for live tabs); 4 handle about double. 2 many whole-bank ad-hoc filters at once, the morning risk meeting; node and hierarchy questions are cheap. 3 live tabs and streams: each open live view holds a connection.

**Not measured yet: VaR under many users.** Next: a concurrency test in the 200K benchmark (50, 100 and 200 users, a realistic VaR mix, against the sharded cluster), then again at 2.3M on a bank server.



[1–6] How these figures were derived: A-4, in the appendix (Concurrency and throughput)

# Storage and deployment

06



# Delta for history and reproducibility, Drishti*net for scale-out

**Partition by business_date; 7 years of time travel**
full run replaces a date; restatement = new commit

**Vector column list<double>, float64**
standard Parquet; a sum reads one column

**Compaction + checkpoints (lake maintain)**
many small intraday commits stay fast

**Sorted by LE and desk; ZSTD + byte-stream split**
the parallel loader: shard-aligned files, a node a run of rows

**Z-order or liquid clustering by book and portfolio** PROPOSED
skip files for a desk's or portfolio's read

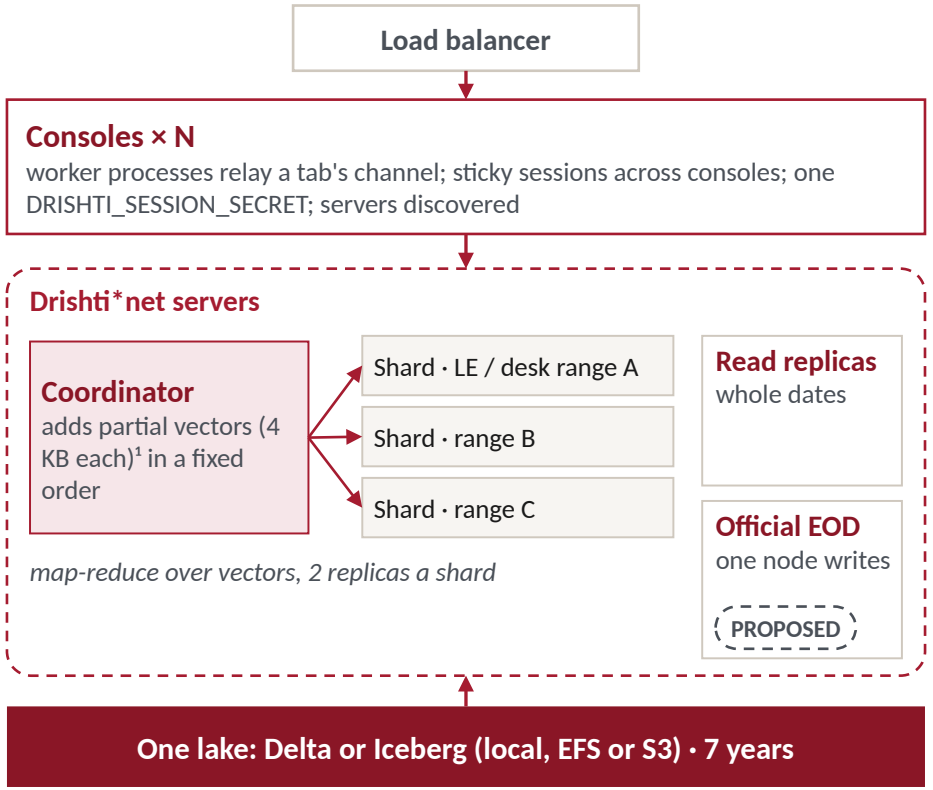
**Published node and portfolio vector table** PROPOSED
BI and peers read sums, not 2.3M rows

**Change data feed on the input tables** PROPOSED
follow intraday deltas downstream

**Delta inputs on S3 (reading)**
the Delta plugin reads s3:// tables

**Iceberg for VaR, intraday and exposure**
equal to Delta to the cent; Delta the default

**Official results to Delta or Iceberg, folder or S3**
append-only; BI's readers follow the format




**To reach 2.3M positions and 15,000 portfolios, still to do**

1 The 200K benchmark, then a 2.3M load on a real server
2 Clustering by book and portfolio (Z-order)
3 The 15,000 portfolio vectors built in a batch
4 Several consoles without sticky sessions
5 The Glue and S3 Tables runs on AWS

Built: VaR sharding, the sorted parallel loader, Iceberg, results to S3



[1] A partial vector: 509 scenarios × 8 B = 4,072 B a node a shard (VAR_SHARDING)

# Delta Lake or Iceberg for VaR: performance and recommendation

**VaR answers are format-independent.** Once a date is loaded, the engine computes from its memory-mapped hot files; the lake format matters only for loading, intraday commits, history and other engines.

**≈ 0.5 ms**[1]
a node's VaR and ES · measured

**≈ 80 ms**[2]
the whole bank at 2.3M positions · estimated

| Where they differ | Delta Lake | Iceberg | Edge |
|---|---|---|---|
| **First load of a date** | replays its log from the last checkpoint | plans from manifests | about the same: both Parquet, Drishti's own readers |
| **Intraday micro-batches (Kafka every 5 min, ≈ 120 commits a day)** | a small log entry per commit, then checkpoints | rewrites its metadata file each commit; ≈ 0.2 GB[3] by year 7 (estimated): needs snapshot and metadata management | **Delta** |
| **Time travel and restatements** | rebuilds a version from its log and checkpoints | jumps to a snapshot | **Iceberg, slightly** |
| **Caches after another day's commit** | equivalent proposed | built | **Iceberg** |
| **Other engines (Spark, Athena, Trino, Snowflake, Databricks)** | strongest with Spark and Databricks | broader multi-engine support | depends on the platform |
| **Correctness** | identical: the equality tests prove Delta = Iceberg to the cent | | same |

## Recommendation

**Delta is the default (product decision), above all with Kafka intraday.** Choose Iceberg where the data platform standardises on it, or for heavy time travel and multi-engine access. Either is a configuration switch.

**Built** VaR, intraday and exposure on Iceberg; official results to Iceberg and S3; equal to Delta to the cent

**Measured (demo)**[4] 14 risk tables, 955,124 rows moved to Iceberg in 579 s

**Not built yet** the 200K and 2.3M benchmarks on both formats; Delta ↔ Iceberg conversion



[1] VarTierBenchmark (VAR_GUIDE §27.3) [2] 4.68 GB of float32 rows at 2.3M ÷ 60 GB/s = 78 ms [3] 120 commits a day × 1,764 dates × ≈ 1 KB [4] CHANGELOG, Iceberg

DEPLOYMENT OPTIONS

# Delta or Iceberg, on premises or on AWS

| | Delta Lake | Iceberg |
|---|---|---|
| **On premises** | VaR inputs from Delta tables<br>Official results: append-only Delta + JSONL<br>Lake maintenance; 7-year floor for risk | VaR, intraday, exposure on Iceberg<br>Official results to Iceberg<br>Maintenance, orphans, 7-year floor |
| **AWS** | Reading Delta tables in S3<br>Official publish to an EFS Delta root<br>Official results to s3:// | Iceberg on S3; AWS scripts<br>Admin → Lakes, disk caches<br>Glue and S3 Tables: first AWS run NOT RUN |

**Above the storage plugins, one product** packs, data contracts, connector names, engines, BI, UI and Drishti*net are the same on Delta and Iceberg; VaR and exposure are equal to the cent on both (VAR BANK USD 112,041,646[1] on the demo data). Delta is the default.

*The AWS deployment and its cost are in the appendix; the companion deck Drishti on AWS (the lake, Delta or Iceberg) has the detail.*



[1] VarFormatEqualityTest: the same lines written to Delta and to Iceberg give the same VaR, to the cent, on the demo data

# Operations and model risk

07

# From the full run to intraday, every business date

**1 Full run lands**

after the close: scenario set, every live trade, hierarchies, portfolios; JSONL into the lake, then the load signal

**2 Date loads**

every row checked; positions derived; nodes summed; the hot file built; currencies built ahead

**3 Official record**

every node of every tree captured from the full run, published append-only, with JSONL

**4 Intraday**

changed trades or positions only, files or 5-minute Kafka windows; answers say intraday; the record stays

| Monitoring, as built | What it shows |
|---|---|
| GET /api/v1/var/status | loaded dates, trades, positions, nodes, load time, rejected, excluded, batch ids, refused dates |
| GET .../var/intraday/streams | topic, state, waiting trades, offsets, last ten windows, newest rejections |
| Provenance on every answer | inputs by system, rejections by code, intraday batches applied |
| Admin pages | Audit, Lakes (maintenance), the connector's Apply now |
| drishti.py var status | the same status from the command line |

## What to check each morning

- The newest date is loaded, from one run (one BATCH_ID)
- Refused rows and refused dates are zero, or explained
- The official record exists; no publishError
- Each system arrived in its expected form and grain
- Limit use: amber 75%, red 90%; the backtest light
- Intraday streams: nothing waiting long; no rejections

**Not checked by the engine today:** the batch's due time and its completeness against the trades booked (planned in the VaR guide); agree them with the pricing-system teams.

# Every number reproducible, every day backtested

## Reproducibility, as built

**Versions on every answer**
inputs' data versions, batch ids, scenario set, method, engine version

**Digests**
the position rules' text and digest, the membership hash

**Append-only official records**
a restatement adds a result version; the earlier is kept

**7-year retention**
vectors with time travel, captures and records; maintenance never goes below

**Recomputable**
the same vectors and engine version give the same number years later; a capture keeps the vector

**Exact arithmetic**
a node's figures equal the lake's to the bit; components add up to VaR

## Backtesting at 99% losses bigger than VaR are breaches

| Green 0–4 | Amber 5–9 | Red 10+ |
|---|---|---|

**Kupiec test** too many or too few breaches?

**Christoffersen test** do breaches cluster?

**Official mode** a node's official history: each day's hypothetical P&L against the day before's VaR

**Replay** today's members over past scenario days: in-sample, labelled not a regulatory backtest

**Demo, replayed**[1] FX desk: 3 breaches in 249 days, green; the bank: 5, amber

**Short histories** limits scale with the days; not a regulatory zone before 250

**Not built, to disclose** age-weighted and filtered methods, ES backtests, 10-day and stressed VaR. The hypothetical P&L is the engine's own; a bank's own hypothetical P&L table replacing it is planned.

Kupiec test (Paul Kupiec, 1995): is the number of VaR breaches (losses bigger than VaR) consistent with 99%, about 2–3 in 250 days?
Christoffersen test (Peter Christoffersen, 1998): do breaches cluster together, a sign the model reacts too slowly when markets turn?
Basel traffic light: breaches in 250 days at 99%: 0–4 green, 5–9 amber, 10+ red.



[1] Replayed on the demo's synthetic data, 249 days (VAR_GUIDE §7); zones by the Basel traffic light for 250 days at 99%

# Independent checks, entitlements, audit, change control

## Independent recomputation

PROPOSED

- A checker recomputes the VaR and ES of the bank and every desk from the files sent, and compares with the engine
- Exits non-zero on any difference: fit for a daily job
- The official record names its inputs' versions and batch ids: a recomputation reads exactly that data

## Entitlements

- Per role and node: positions, or aggregate only (drishti.var.access)
- A scope is computed over the trades you may see; the rest counted
- Masks apply; answers cached per view, never shared across rights
- Saving baskets and captures needs the token scope risk:write

## Audit

- Every headline, capture, basket change, reload and official run (Admin → Audit)
- A headline's entry: date, data version, view, VaR, ES, trades, hidden, members' digest, time
- Reloads and official captures: administrators only; official numbers kept 7 years

## Change control of settings

PROPOSED

- Window, confidence, rank rule, currencies, pins: in application.yaml, shown by GET /status
- Every answer is labelled with its confidence and window; per-request windows are never official
- Pins read at start: apply from the next date loaded; provenance records them

*PROPOSED marks the practice: a daily independent job, and comparing a setting change per request before making it; the mechanisms are built.*

# Editions

08

# One file makes a focused product: Drishti Risk, Market or World

### An edition is one configuration file

`config/editions/<name>.yaml`

It sets the packs, the commands and words people may use, the menus and features, BI's datasets, Home and its landings, help, the Admin pages and the branding. The site's own settings still win, key by key.

```
# config/editions/risk.yaml, abridged
edition: risk
title: Drishti Risk
packs: { enabled: [core, market-data, trading,
       counterparty-risk, market-risk], allow-only: true }
commands: { allow: [VAR, VRES, EXP, NSET, NSMTM, …] }
home: { landing: desk, landings: [desk, command, map],
       landing-by-role: {} }
branding: { product: Drishti Risk }
```

| | Edition | Lens |
|---|---|---|
| **Set by** | DRISHTI_EDITION, then a restart | a person's link: Open Drishti Risk |
| **Offered by** | the administrator, every server | DRISHTI_LENSES on a full instance |
| **Scope** | the whole instance | one person's session, no restart |
| **Packs** | only the edition's | all stay loaded |
| **Secures?** | refuses packs, hides commands | hides; it does not secure |

**In Views and on Home:** Open Drishti Risk; inside one, Leave … first, then Switch to … the others.

PLANNED

**Role-based access:** home.landing-by-role is read and wins when set; who sees a lens (lens.roles) is a hook, not wired.

### Drishti Risk

VaR and exposure from the bank's own data; stress left out

**Landings:** desk · command · map

### Drishti Market

curves, surfaces and fixings that feed risk and valuations

**Landings:** desk · explorer · quality

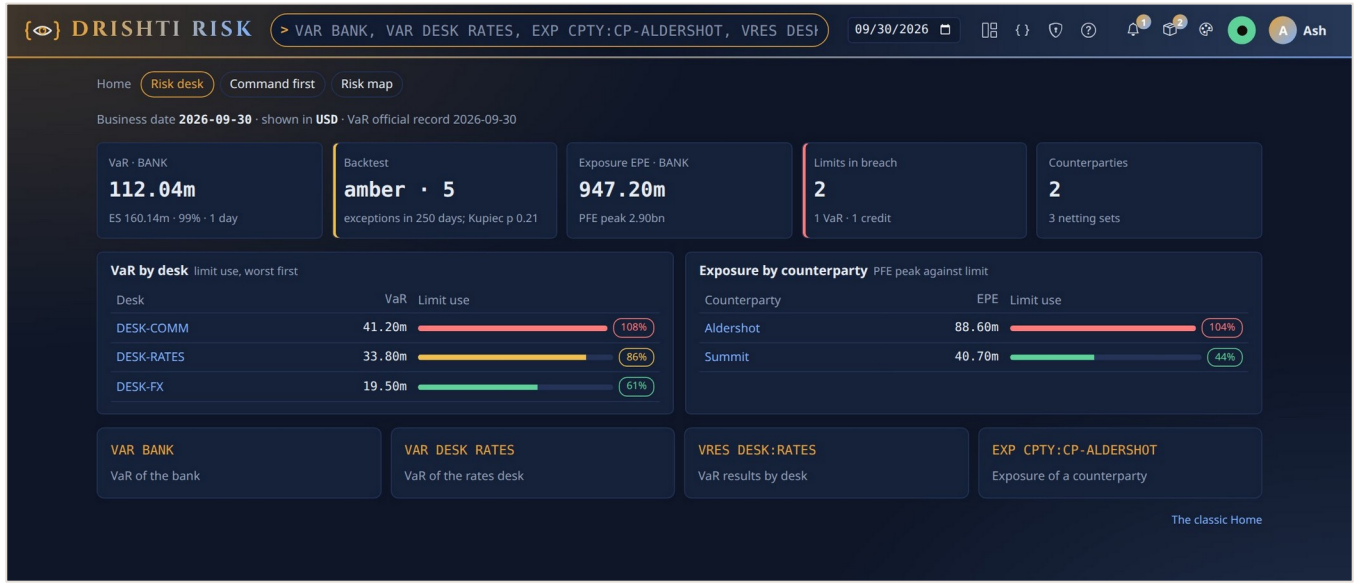
### Drishti World

economies, markets, companies and funds from public data

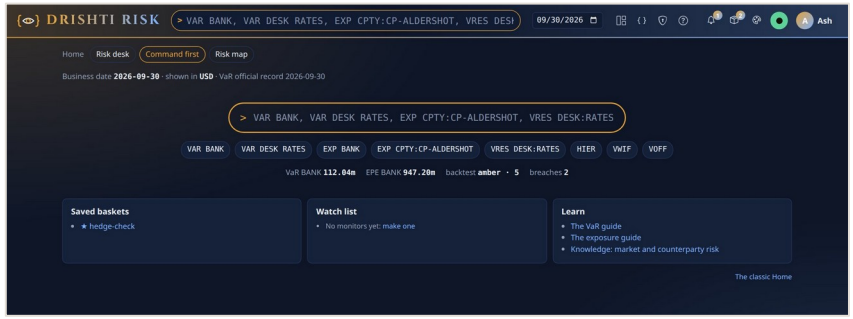
**Landings:** desk · explorer · map

*The engines are unchanged under an edition: VaR and exposure of any scope, sharded or not, equal to the cent (the risk suites run again with the edition on).*

# Drishti Risk's three landings: desk, command and map




*A · Risk desk (the default): VaR, backtest, exposure, limits in breach; desks and counterparties*




*B · Command first: the command line, chips, baskets*




*C · Risk map: a hierarchy with VaR and limit use, a heat map, Today*

## Every figure is the engines' own answer

- The landings read the console's VaR and exposure routes (scope, tree, intraday streams, official records, baskets); no new store, no copy of a figure
- The choice of landing is kept per person; a role can be given its landing (home.landing-by-role)
- Pictures from the browser tests' stand-in server: VaR 112.04m and EPE 947.20m are the demo bank's 2026-09-30 figures; the desk and counterparty rows are stand-ins

# Drishti Market: the market data that feeds VaR

**Why a VaR user cares.** VaR is read from scenario P&L that the pricing systems build from the bank's curves, vol surfaces and fixings. When that market data is stale or broken, so is the VaR built on it. Drishti Market shows that data, and checks it.

| Asset class | Commands | What it holds |
|---|---|---|
| Rates | `CRV · REPO · IRV · CPV · FIX · INF · INFC` | OIS and projection curves, repo curves, swaption and cap vols, fixings, inflation |
| FX | `FX · FXF · FXV` | spot, forward curves, vol surfaces |
| Credit and bonds | `CDS · BND` | credit curves and spreads, bonds |
| Equity and commodities | `EQ · EQX · DIV · EQV · CMD · CMDC · CMDV` | equities, indices, dividends, equity vols; commodities, their curves and vols |
| Correlation | `CORR · CORRC · EQB` | correlations and baskets |

## One file: config/editions/market.yaml

- the market-data pack and core (issuers, holiday calendars), allow-only: its 22 commands, nothing else
- no trades and no VaR: VAR, EXP and the risk and trade commands are hidden
- Home: Market desk (the default), Explorer, Quality
- the engines are unchanged: nothing in them moved for the edition

## For a VaR user: a lens beside Drishti Risk

- On a full instance, drishti.lenses: risk,world,market offers Open Drishti Market next to Open Drishti Risk; inside one, Switch to … opens the other in one step, no restart
- A lens hides; it does not secure: what a person may read stays with roles, entitlements and masks

# Market data quality: the checks behind VaR's inputs

**Checked on the business date, worst first; each item opens its command.** Tiles: Critical, Warnings, Clean (the share of items with no exception); a Checks run card gives each check's items and flags.

### 1 · Stale *warning*

- a curve's AS_OF, or the date its source holds it for, older than the business date by more than quality.stale-days (2 business days; weekends and holidays skipped)
- curves and surfaces one by one; every other kind by its latest load

### 2 · Gaps: missing tenors *warning*

- a curve missing a tenor that every other curve of its type has (a type of three or more)
- IR curves by CURVE_TYPE; repo, credit, FX forward and inflation curves by kind

### 3 · Discount factors and forwards *critical · warning*

- critical: a discount factor that does not fall with maturity, or is outside (0, 1]
- warning: a negative forward rate in FORWARDS

### 4 · Vol surfaces: sanity *critical*

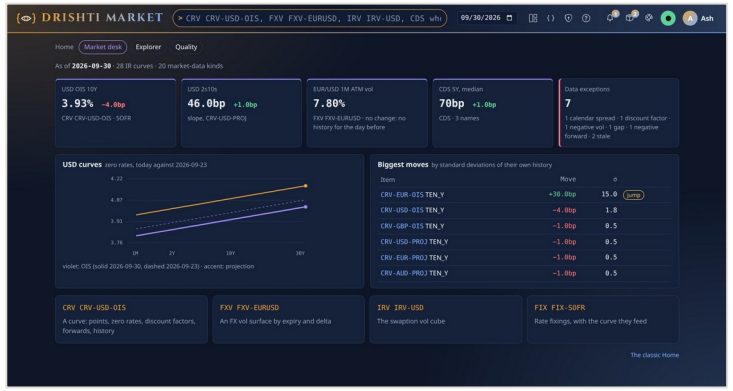
- a vol at or below zero (risk reversals and butterflies may be negative)
- ATM total variance ($vol^2 \times T$) falling with expiry, on the FX and equity surfaces

PLANNED

### Impact on VaR

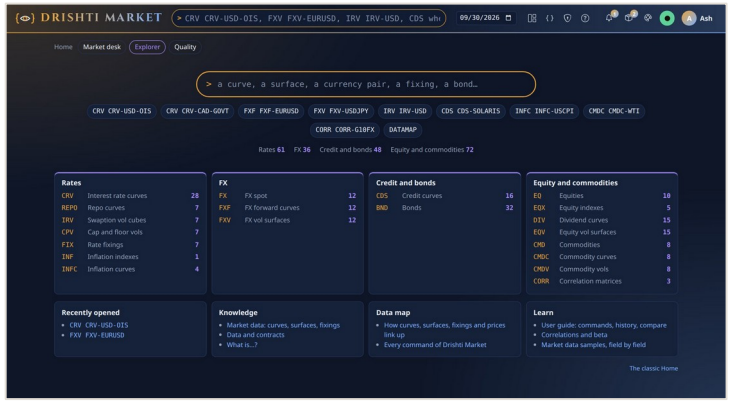
- who uses each flagged point and the VaR at stake; jumps over n σ as a quality check; two sources disagreeing (the lake holds one source per kind today)
- Today the Quality landing says so in place of the figure: impact on VaR and valuations comes later

# Drishti Market's three landings: desk, explorer and quality



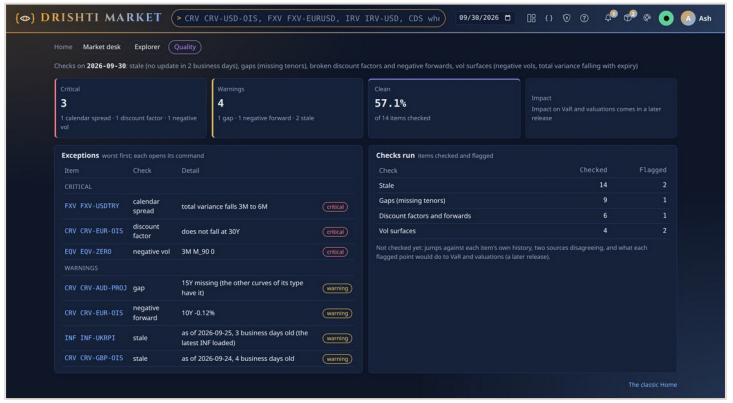

## A · Market desk (the default)

- tiles: USD OIS 10Y, USD 2s10s, EUR/USD 1M ATM vol, CDS 5Y median, data exceptions
- USD curves, today against a week before
- biggest moves over 21 business days, in σ of each item's own moves




## B · Explorer

- a big command line and the edition's chips
- counts by asset class: rates, FX, credit and bonds, equity and commodities
- the commands by asset class; recently opened; Knowledge, Data map, Learn




## C · Quality

- the four checks, worst first: stale, gaps, discount factors and forwards, vol surfaces
- Critical, Warnings, Clean; the Checks run card
- the impact on VaR: a note, not a figure (planned)

**Every figure is the pack's own answer.** Worked out in the console from the market data's search, documents and history; read-only, cached two minutes per person. A day-over-day change only from real data on both days; otherwise the tile says so.

*Pictures from the browser tests' stand-in server, with known problems seeded on purpose; the demo data's checks are all clean.*

# Drishti World: the macro context for risk

**Why a risk user cares.** A VaR or exposure move has a story outside the bank: an economy's growth, inflation or debt, the day's news, a company's filings, an entity's owners, a fund's holdings. Drishti World puts that public context one command away.

| Pack | Commands | The context it gives |
|---|---|---|
| Economies | `ECON · MACRO · FCST · LABR · FISC · ECG · REGE · JUR · PARTY · OFFC · ELEC · REGN · SOCI` | economies and regions: growth, inflation, debt, labour, fiscal; jurisdictions, parties, elections |
| News and markets | `TOPIC · NEWS · CO · QUOTE` | news by topic; companies; a market snapshot of quotes |
| Companies | `USEQ · IDX · SIC · EDGAR · FILING · FACT · PXH` | US companies, indices, sectors, SEC filings and facts, price history |
| Legal entities | `LEI · LEIG · LEIR · LEIC · LEIM` | GLEIF: entities, groups, parents and children |
| Funds | `FUND · HELD · TRACK` | funds and ETFs, what they hold, the index each tracks |

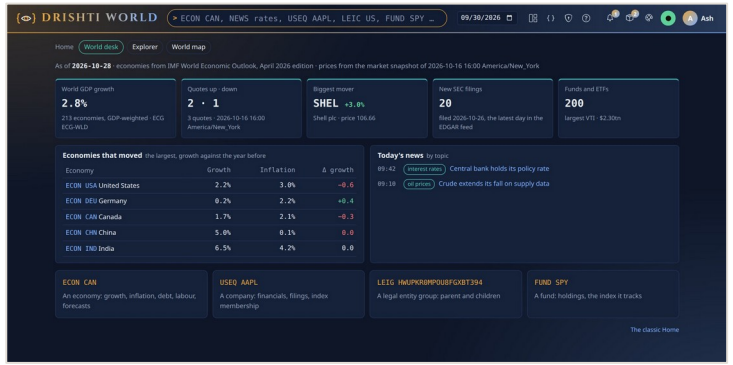
## One file: config/editions/world.yaml

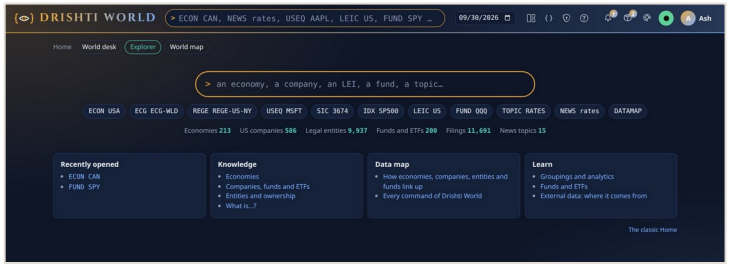
- the public-data packs, allow-only: world-economy, news-and-markets, us-equities, gleif, funds
- their 32 commands; VAR, EXP and every risk command hidden
- Home: World desk (the default), Explorer, World map
- a lens beside Drishti Risk: Switch to … in one step

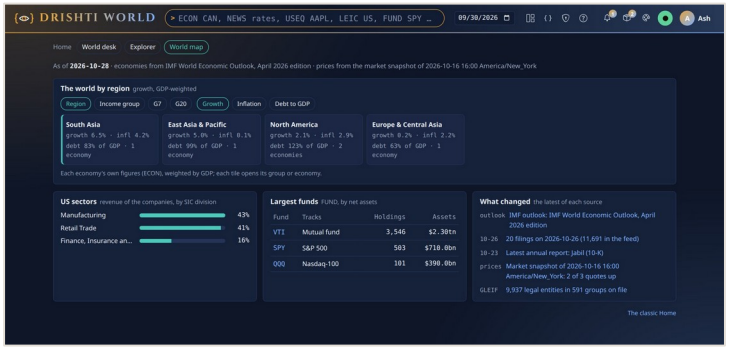
## Public data, inside the bank

- The IMF figures and the market prices in these packs are shown inside the bank; showing Drishti World outside it needs the product owner's decision first

# Drishti World's three landings: desk, explorer and map




## A · World desk (the default)

- tiles: World GDP growth, quotes up and down, the biggest mover, new SEC filings, funds and ETFs
- economies that moved: the twelve largest
- today's news by topic

## B · Explorer

- a big command line and the edition's chips
- counts: economies, US companies, legal entities, funds and ETFs, filings, news topics
- recently opened; Knowledge, Data map, Learn

## C · World map

- by region, income group, G7 or G20; by growth, inflation or debt to GDP, GDP-weighted
- US sectors; the largest funds
- what changed: each source's latest

**How it reads.** One thin read-only route and the server's own search grammar (ECON where GDP_USD_BN > 0 order by GDP_USD_BN desc limit 12); a command the person's edition or lens does not show is refused (DRS-4023). Each panel paints when its answer arrives.

*A teal accent, so World reads apart from Risk at a glance. Pictures from the browser tests' stand-in server.*

# Regulatory reporting

09

# Canada and the US: the reports that need VaR or ES

**Today.** Canada: FRTB since November 2023; every bank reports the standardised approach, internal models need OSFI's approval. US: VaR and stressed VaR (Basel 2.5) in force; an ES-based re-proposal of March 2026 is not final.

| Canada (OSFI) | How often | VaR or ES it needs |
|---|---|---|
| **BCAR 90.010** | Quarterly | standardised for all; ES 97.5% and SES, latest and 60-day, on internal models |
| **Pillar 3 MR1, MR2** | Quarterly | MR2: ES, SES, DRC, latest, average, high, low; exceptions against 99% VaR |
| **Internal-model tests** | Daily | 1-day VaR 99% and 97.5% against actual and hypothetical P&L; the PLA test |
| **Annual report** | Yearly | trading VaR, 1 day 99%, by risk type: year-end, average, high, low |
| **ICAAP, stress tests** | Yearly | no prescribed VaR or ES |

| US (Fed, OCC, FDIC) | How often | VaR or ES it needs |
|---|---|---|
| **Market risk rule** | Daily | VaR 99% 10-day and stressed VaR; 60-day and 12-week averages × 3 to 4 |
| **FFIEC 102** | Quarterly | VaR and stressed VaR, latest and averages; public |
| **§217.212** | Quarterly | high, low, mean, period-end VaR and stressed VaR; by risk type; backtest |
| **Volcker metrics** | Quarterly | per desk, daily: limits and use, VaR 1-day 99%, P&L attribution, positions, volumes |
| **FR Y-9C, FR Y-14Q** | Quarterly | market-risk RWA (HC-R item 27); the market shock: not VaR |

**A Canadian bank with a US holding company.** It files both sides: the US holding company follows the US capital rule like a US bank holding company; FFIEC 102 applies at $1 billion of trading assets and liabilities, Volcker at $20 billion across the bank's US operations.

# The building blocks, and what Drishti has today

| Building block | Needed by | Status | Drishti today |
|---|---|---|---|
| **VaR 99% and ES 97.5%, 1 day** | MD&A, Volcker, every backtest | | the bank to a trade; any node or trade set |
| **By desk and legal entity** | all; diversification in most | | every hierarchy node, with diversification |
| **VaR limits and their use** | Volcker | | a limit per node; use in every official record |
| **Kept and reproducible** | all | | official runs of every node, 7 years, versions |
| **Backtest: hypothetical P&L** | all | | every node: exceptions, traffic light, tests |
| **Backtest: actual P&L, 97.5%** | US rule, OSFI internal models | NOT BUILT | needs the bank's daily P&L feed |
| **10-day VaR, stressed VaR** | US rule, FFIEC 102, §217.212 | PLANNED | in the plan |
| **High, low, average, 60-day** | MD&A, FFIEC 102, §217.212 | NOT BUILT | from the official history Drishti keeps |
| **VaR by risk type** | MD&A, §217.212, FFIEC 102 | NOT BUILT | needs P&L vectors by risk class |
| **Other limits, P&L attribution** | Volcker | NOT BUILT | P&L explain designed, not built |
| **Standardised approach** | BCAR 90.010, Pillar 3 MR1 | NOT BUILT | contract designed (FRTB SA) |
| **ES by liquidity horizon, PLA** | banks on internal models | NOT BUILT | with NMRF and DRC |

**Reading it.** A Canadian bank on the standardised approach reports VaR in its annual report; its US holding company needs the full VaR and stressed VaR set. Built rows carry no pill.

# From the official run to the filing, reproducible as of a date

| Inputs | Official run | Computed measures | Report layer | Filing |
|---|---|---|---|---|
| ▪ trades, P&L vectors, scenario sets | ▪ end of day, every node | ▪ VaR, ES, backtest, diversification | ▪ BI reports and paginated reports | ▪ the bank's own filing channels |
| ▪ hierarchies, limits | ▪ inputs' versions and batch ids | ▪ 10-day and stressed VaR<br>PLANNED | ▪ extracts: Excel, CSV, PDF | ▪ the bank's review and approval |
| ▪ daily P&L from product control<br>NOT BUILT | ▪ a restatement is a new version | ▪ period statistics, averages<br>NOT BUILT | ▪ the published official table | ▪ Drishti does not file |

## Example: a quarter's US VaR figures

- the official records of the quarter's business days, each node's VaR at its own result version
- 10-day VaR: the latest and the 60-day average × the multiplier from 250 days of exceptions
- stressed VaR: the latest and the 12-week average; high, low, mean and period end by risk type
- laid out as the report's table; exported for the bank's filing

## Reproducible as of a date

- each record names its inputs' versions, batch ids and position rules
- the members of every node kept with the record
- append-only, 7 years, Delta or Iceberg
- recomputed from the same vectors and engine: the same number

# Roadmap and rollout

10

ROADMAP

# What is built, and what comes next

## Built · unlabelled

**Any input: PVs or P&L, trade or position**
files and Kafka; mixed per system

**Intraday: files and Kafka micro-batches**
new – old; equal to one run to the cent

**Any reporting currency; five built ahead**
scenario FX; exact per-currency totals

**Official records, append-only, 7 years**
every hierarchy; Delta or Iceberg, folder or S3

**Iceberg, a full alternative to Delta**
VaR, intraday and exposure equal to the cent

**Hierarchies configured by columns**
limit letters, var-limit, the picker

**VaR sharded across a cluster; the Java loader**
two replicas a shard; JSONL to Delta or Iceberg, shard-aligned

**Editions: Drishti Risk, Market and World**
one file each; lenses on a full instance

**Computed attributes**
derived MtM and checks, never stored

## Not built yet

**10-day and stressed VaR** — PLANNED
sized in the capacity chapter; regulatory phase 1

**ES backtests** — PLANNED
VaR backtests are built

**The 200K benchmark, then 2.3M on a real server** — PLANNED
single server against a sharded cluster

**Arrow to the browser** — PLANNED
large tables and charts, server to console to browser

**A client-side renderer** — PLANNED
other applications paint Drishti screens from its view JSON

**Delta ↔ Iceberg table conversion** — PLANNED
today a lake is built in either format

**Delta in-place schema evolution** — PLANNED
Iceberg evolves in place today

**Unity Catalog; plain tables** — PLANNED
Delta through Unity Catalog, opt-in per connector

**Glue and S3 Tables on AWS** — NOT RUN
code written; first live run on an account

PROPOSED **At scale:** lake clustering by book and portfolio (Z-order or liquid clustering); a published table of node and portfolio vectors, built in an overnight batch.

**Later:** QLRisk what-if pricing; age-weighted methods; cubes. **Known gap:** live tabs across consoles need sticky sessions. Regulatory phases: next slide.

# Regulatory reporting in three phases: next, later, after that

## 1 · The measures
next

NOT BUILT

**Builds**

- 10-day and stressed VaR
- a daily actual and hypothetical P&L feed
- backtests: 99% and 97.5%, both P&Ls, multiplier
- high, low, average, 60-day; CAD records

**Unlocks**

US rule figures; FFIEC 102 and §217.212 inputs; the annual report's statistics

## 2 · The report packs
later

NOT BUILT

**Builds**

- VaR by risk type
- BI reports and extracts in each report's layout
- Volcker metrics per desk

**Unlocks**

annual report VaR table, Pillar 3 tables, FFIEC 102, §217.212, Volcker

## 3a · FRTB standardised
after that

NOT BUILT

**Builds**

- sensitivities-based method
- default risk charge, residual risk add-on

**Unlocks**

BCAR 90.010, Pillar 3 MR1; the US standardised approach if adopted

## 3b · FRTB internal models
after that, only on approval

NOT BUILT

**Builds**

- ES by liquidity horizon, stressed calibration
- PLA test; NMRF and SES; DRC

**Unlocks**

BCAR 90.010 section A; Pillar 3 MR2; model approval

**Why 3a before 3b.** Every Canadian bank reports the standardised approach, modelled or not; internal models serve only a bank that applies.

**The bank supplies:** daily actual and hypothetical P&L; 10-day and stress-window P&L vectors; P&L by risk class; desk definitions and its limit catalogue; sensitivities; real-price observations for 3b.

# A phased rollout with a parallel run

### 0 Prepare

- Environment: Delta or Iceberg, on premises or AWS
- Settings and entitlements agreed
- The 200K benchmark, then 2.3M on the target server

### 1 Pilot desk

- The desk's systems onboarded
- Daily full runs and official records
- Parallel run against the existing VaR

### 2 A legal entity

- All its desks and systems
- LEGAL tree and limits
- Intraday switched on

### 3 The bank

- Every system, every tree
- ≈ 15,000 portfolios; ≈ 700 limit letters
- BI reads the official results

### 4 Cutover

- Sign-off
- Drishti's records become the numbers of record
- The existing VaR retired

## Parallel run and reconciliation

- Each node, each date: the existing VaR beside Drishti's official record (the var-official table)
- Explain every difference by its cause: inputs (refused, excluded, unknown ids), method (window, confidence, rank rule), FX (scenario FX, not today's rate), membership (overlaps counted once), model
- Track the backtest light on the official history as it builds up

## Sign-off criteria

PROPOSED

- Agreed tolerance per node; every difference explained
- No unexplained refusals over an agreed run of days
- Model validation of the method settings and the controls
- Runbook rehearsed: a restatement, a failed publish, a restart
- Backtest light not red; entitlements and audit reviewed

*A proposed plan: generic, no dates. Each phase uses built capabilities; the 10-day and stressed measures are not built yet.*

# Appendix

A

# What a pricing system sends

**Every record** JSONL: one JSON object a line, UTF-8, .jsonl or .jsonl.gz; files or Kafka

| Field | Values | Rule |
|---|---|---|
| `VALUE_TYPE` | PV or PNL | required |
| `GRAIN` | TRADE or POSITION, with its id | required |
| `CURRENCY` | ISO code; PVs in their own | required |
| `SOURCE_SYSTEM` | the pricing system that sent it | required |
| `SCENARIO_SET_ID` | the date's set (demo: HS-509) | all, in order |
| `BATCH_ID · RUN_KIND` | FULL or INTRADAY | one full run a date |
| `COMPUTED_AT` | UTC instant | the latest wins |
| `LEGAL_ENTITY · DESK · TT_BOOK` | where it sits | needed to load |
| `LIMIT_LETTER` | a level column of a hierarchy | absent: tree skipped |
| `BASE_PV · SCENARIO_PV` | the PV form | failed: null |
| `PNL (+ PV)` | the P&L form, USD | PV makes FX exact |
| `TRADE_STATUS · MATURITY_DATE` | live trades count | CANCELLED: cancel |

### Files

the full run, and small intraday batches; lake build, then the load signal

### Kafka

JSONL text per message, one or more records; one lake batch per 5-minute window

### Intraday and cancels

- RUN_KIND INTRADAY, its own BATCH_ID, a later COMPUTED_AT, the morning's scenario set
- A cancel: TRADE_STATUS CANCELLED, the vector null
- Same form and grain as the system's full run

```
# optional per-system pin (read at start)
drishti.var.inputs:
  SYS_A:   {form: pv,  grain: position}
  SYS_B:   {form: pnl, grain: trade}
  default: {form: any, grain: any}
```

*A null scenario PV refuses the whole trade (DRS-5206), never zero-filled. Field names are the as-built contracts'; older files are brought to the record with drishti.py convert --set ….*

# A checklist for each pricing system

1. **Choose form and grain**
   PVs or P&L, by trade or by position: one form and one grain per system per date
2. **Map the record's fields**
   VALUE_TYPE, GRAIN + id, CURRENCY, SOURCE_SYSTEM; LEGAL_ENTITY, DESK, TT_BOOK; status, maturity
3. **Price over the date's scenario set**
   every scenario in the set's order; a failed one null, never zero
4. **Scenario FX for its currencies**
   send fx-scenario, or let Drishti derive it from the FX history
5. **Pick the route**
   JSONL files for the full run; files or a Kafka topic for intraday
6. **Set the optional pin**
   drishti.var.inputs: a record that disagrees is refused (DRS-5209)
7. **Recompute independently**
   from the sent files, compare bank and desk VaR and ES with the engine
8. **Clear the refusals**
   provenance lists every refused row by code; aim for none before go-live

PROPOSED

**Ready when** the system's records load with no refusals, the independent recomputation agrees to the cent, and How this number was built shows the system with its form and grain.

# When the data's structure changes

## A new field in the source data

No schema change. Each record's full JSON document is kept in the `doc` column, so a new field is readable at once; older rows simply lack it.

## A new promoted column

**Iceberg:** in-place schema evolution by field id: add, rename, reorder, widen.

**Delta:** in-place add of a nullable column, old files read as empty, optional backfill from the documents. Until then, adding one to an existing table rewrites the affected dates.

On upgrade: unedited connector files follow the pack; edited ones gain its new keys and columns, edits kept.

PLANNED Delta in-place add

## History is never rewritten

Time travel shows each version as it was; official records are append-only. Adding a field never invalidates the lake, and a figure of record can still be recomputed from its inputs.

```
# before: the source sends A and B
{"id":"T1", "doc":{"A":1,"B":2}, "A":1}
# after: the source adds C; nothing to migrate, C is in doc at once
{"id":"T2", "doc":{"A":3,"B":4,"C":5}, "A":3}
# C promoted to a column: new rows carry it, old files read it as empty (null)
```

# Hardware plan with 39% headroom

## 1-day, 10-day and stressed VaR at 2.3M positions

provisioned = need ÷ 0.61

| Item | No. | Need, each | Provisioned, each | Large (2×) |
|---|---|---|---|---|
| VaR servers (EOD, intraday, 2 UI)[1] | 4 | one machine type | identical, any role | 5 (+ history) |
| vCPU | – | 32 | 64 | 96 |
| RAM | – | 151 GB | 256 GB | 512 GB |
| Heap (-Xmx) | – | 21 GB | 36g | 60g |
| VaR memory budget | – | 14,000 MB | 23,000 MB | 46,000 MB |
| Local NVMe (tier.dir) | – | 1.18 TB | 2 × 1.9 TB | 4 × 1.425 TB |
| Console hosts[2] | 2 | 25 pages/s | 4 vCPU, 8 GB, 4 workers | the same |
| Lake, 7 years (object store)[3] | 1 | 48.2 TB | 79 TB; 158 with DR | 158; 316 with DR |
| Shared state (EFS or NFS)[3] | 1 | 0.3 TB | 0.5 TB | 0.5 TB |
| Network, a date's first open[4] | – | 20.9 GB, ≤ 0.4 GB/s | ≥ 10 Gbit/s | 25 Gbit/s |
| Intraday traffic[5] | – | 17.5 MB a window | < 1 MB/s a server | < 2 MB/s |
| Kafka PV topic, 7 days[6] | – | 5.5 GB a day; 39 GB | 64 GB a replica | 128 GB |

### Does one server hold a date?[7]

Yes. All four sets need 151 GB: one 256 GB server per role, 512 GB at 2×.

Sharding is needed beyond ≈ 5M positions (a date takes 4–10 min to open) or ≈ 10M (over 1 TB of RAM).

### Assumptions to confirm

ASSUMPTION

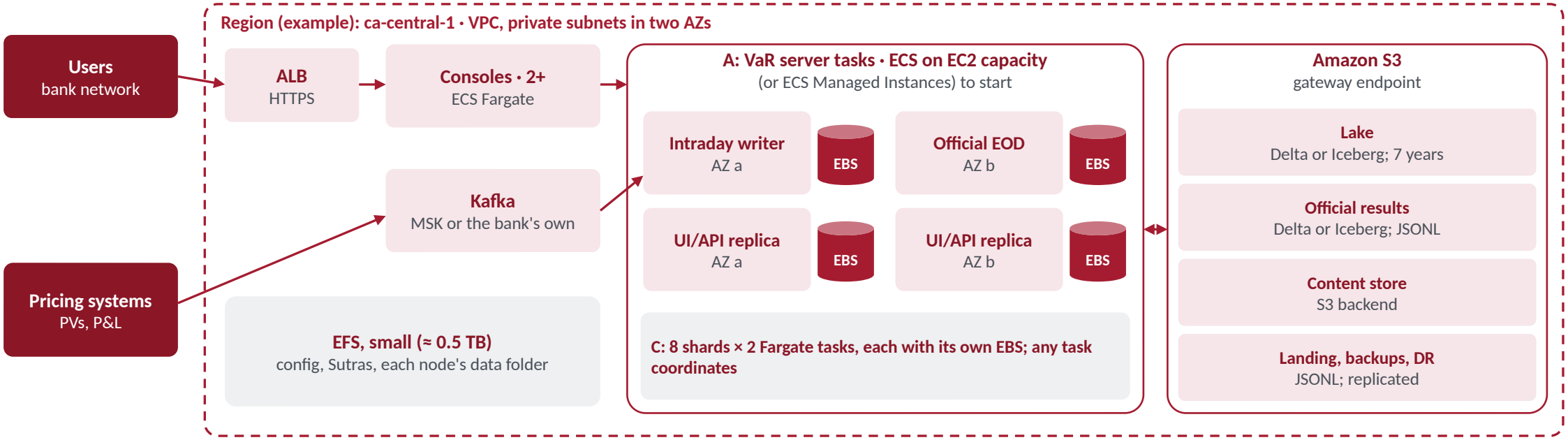
- 50 of 200 users at once; peaks 5×
- 10-day by overlapping returns; stressed 1- and 10-day
- Intraday 230k rows a day, 120 windows
- 5,700 nodes; 2% restated; a DR copy
- Kafka 24 KB a record, kept 7 days

**AWS example:** m6id.16xlarge (64 vCPU, 256 GiB, 2 × 1.9 TB NVMe, 25 Gbit/s); large r6id.24xlarge; Fargate consoles; S3.
**On premises:** 2 × 16 cores (64 threads), 256 GB on every memory channel, 2 × 1.92 TB NVMe, 25 GbE; an S3-compatible object store.



[1–7] How these figures were derived: A-4, in the appendix (Hardware plan)

# Drishti on AWS: ECS, Fargate, S3 first, as little EFS as possible




| VaR tier | Shape (full case: 1-day, 10-day, stressed) | Fits, with 39% headroom[2] | Status |
|---|---|---|---|
| A · ECS on EC2 capacity[1] | 4 × m7i.16xlarge (64 vCPU, 256 GiB), 2 TB EBS gp3 each; the rest on Fargate | 151 GB need → 256 GB machine | |
| B · √10 10-day on Fargate[3],[5] | 4 Fargate tasks of 32 vCPU · 244 GB (the largest), 1.1 TB gp3 | 94 GB → 155 GB: memory yes; CPU to measure | PLANNED |
| C · sharded on Fargate[4] | 8 shards × 2 tasks of 8 vCPU · 44 GB, 250 GB gp3 each, one S3 lake | 26 GB a task → 43 GB | |

**Recommendation:** Fargate for the consoles and lake tasks; A for the VaR servers to start (buildable today on Delta or Iceberg); C, Fargate everywhere, with VaR sharding (built; to run on Fargate). Hot files on an EBS gp3 volume per task: disposable, rebuilt from the lake. EFS only for what must be shared.

PROPOSED



[1–5] How these figures were derived: A-4, in the appendix (AWS: the VaR tier)

# AWS cost estimate for a large bank

ASSUMPTION

| Monthly, year 7, 39% headroom[1] | 1-day only | Full | Full, a year |
|---|---|---|---|
| VaR servers (4, EC2 capacity)[2] | $9,124 | $10,498 | $125,976 |
| EBS gp3 for hot files (4)[2] | $898 | $1,320 | $15,840 |
| Consoles (2, Fargate)[3] | $317 | $317 | $3,805 |
| Lake loads, maintenance (Fargate)[3] | $31 | $31 | $369 |
| S3: lake, results, content store[4] | $258 | $752 | $9,021 |
| DR copy (S3 replication)[4] | $252 | $753 | $9,038 |
| EFS and its backup[5] | $190 | $190 | $2,280 |
| Amazon MSK (3 brokers)[5] | $531 | $531 | $6,375 |
| Load balancer (ALB)[5] | $31 | $31 | $371 |
| NAT and interface endpoints[5] | $173 | $173 | $2,074 |
| Data transfer[5] | $92 | $92 | $1,104 |
| CloudWatch logs and metrics[5] | $217 | $217 | $2,609 |
| **Total, on-demand** | $12,119 | **$14,910** | **$178,914** |
| with a 1-year Savings Plan[6] | $9,596 | $12,017 | $144,201 |
| with a 3-year Savings Plan[6] | $7,443 | $9,553 | $114,637 |

*Prices: AWS public on-demand list prices for the example Region, read 2026-10-27 (ASSUMPTION); quantities from the capacity slides (ESTIMATED). Verify before quoting.*

## The VaR tier: A, B or C[7]

servers and EBS, $ a month

| Option | On-demand | 3-yr plan | + Spot |
|---|---|---|---|
| A: EC2 capacity | $11,818 | $6,618 | – |
| B: √10 on Fargate | $8,633 | $5,200 | – |
| C: sharded, 24 × 7 | $7,373 | – | – |
| **C: scale-in off-hours[8]** | $5,050 | $2,966 | **$2,785** |

### Honest on both sides[8]

- One full copy costs more on Fargate: 8 tasks $3,640 against one EC2 server $2,954 a month
- C wins by needing 2 copies, not 4, scaling replicas in off-hours, Spot for second replicas, growing by adding tasks
- C beats A once A needs more than 2.5 full copies; A needs 4

### Recommendation

**A to start; C once sharding runs on Fargate:** the cheapest ($2,785 against $6,618 a month with 3-year plans) and fully ECS-native.



[1–8] How these figures were derived: A-5, in the appendix (AWS cost)

# When something goes wrong, and the runbook

| Situation | What the engine does | What the operator does |
|---|---|---|
| **Bad rows (DRS-5201, 5206, 5207, 5208, 5209)** | refuses each row, names it; never zero-fills | fix the rows, restate the date |
| **A trade twice on a date (DRS-5202)** | does not load the date | restate with one row per trade |
| **Two runs or sets on a date (DRS-5203)** | answers the date before, with a warning | restate as one run; signal |
| **Second form or grain from a system (DRS-5210)** | refuses: never a double count | fix the system's output |
| **A date over the memory budget (DRS-5204)** | 503; never loads part of a date | raise memory-budget-mb or heap |
| **Restatement** | reloads; next result version; old kept | rewrite the whole date, new BATCH_ID |
| **Publish fails (disk, root)** | records publishError; capture stands | fix the root; capture again |
| **Damaged or old hot file** | reads the lake, rebuilds the file | nothing |
| **Restart, or a Kafka replay** | rebuilds from the lake; writes nothing twice | nothing |

**Runbook**

**1 Check status and provenance** → **2 Name the code** → **3 Fix at source** → **4 Restate the whole date** → **5 Signal; verify the version**

*The engine never guesses: a bad row is refused and named; a date that cannot load as one run is not loaded, and the day before is answered with a warning rather than a wrong number.*

GLOSSARY

# The terms used in this deck

| Term | Meaning |
|---|---|
| **VaR** | the loss not exceeded at the confidence: 99% = minus the 6th worst of 509 |
| **ES** | expected shortfall: minus the mean of the 13 worst P&Ls at 97.5% |
| **Scenario** | a historical day's market move, applied to today's trades |
| **Scenario set** | the scenario days of a date (HS-509), oldest first |
| **Scenario PV** | a trade's or position's value under one scenario, in its own currency |
| **Path P&L** | the P&L vector: one P&L per scenario (509 numbers) |
| **Form · grain** | PV or P&L · one record a trade, or one a position |
| **Scope** | any set of trades or positions: bank, LE, desk, book, portfolio, rule, basket |
| **Node** | a point of a hierarchy; its vector precomputed |
| **Limit letter** | a node of LL, a hierarchy by columns; limit from var-limit |
| **Position** | trades grouped by the pack's rules, or sent whole |
| **Portfolio** | a stored scope: HIERARCHY, RULE or BASKET |

| Term | Meaning |
|---|---|
| **Component VaR** | a part's share of VaR (Euler); parts add up to it |
| **Incremental VaR** | VaR(scope) − VaR(scope without the part) |
| **Marginal VaR** | component VaR per million of size |
| **Diversification** | the sum of standalone VaRs minus the VaR |
| **Scenario FX** | each scenario's exchange rates, used to convert |
| **Reporting currency** | the currency an answer is in; five built ahead |
| **Hot file** | a date's vectors, memory-mapped from local disk |
| **Official record** | a node's end-of-day VaR and ES, append-only |
| **Result version** | 1, 2, 3 … as a date is restated |
| **Provenance** | how a number was built: inputs, versions, refusals |
| **Breach (exception)** | a day whose loss exceeds the VaR tested against |
| **Headroom 39%** | provisioned = need ÷ 0.61 |

# How the numbers were derived

## How it scales

[1] A row of 509 scenarios: 509 × 8 B = 4,072 B in float64 (the lake, the sums), 509 × 4 B = 2,036 B in float32 (the hot file's rows) (VAR_GUIDE §27.1).

[2] 21.4 MB a reporting currency a date: measured on the demo lake, 8,566 trades (VAR_GUIDE §15.11).

[3] Measured on one workstation (24 threads, 61 GB, NVMe): VarTierBenchmark, 100,000 trades × 5 dates (VAR_GUIDE §27.3); VarScaleBenchmark, 1,000,000 trades in memory (§27.2); the demo date's load, 9,572 trades, on the server (§26.7). 60 GB/s = 203.6 MB of float32 rows in 3.4 ms; 70 GB/s = 1M × 509 × 8 B in 58 ms.

## Sizing

[1] Vectors: 2.3M positions (ASSUMPTION, the bank's volumes) × 509 scenarios × 8 B = 9.37 GB a date in float64; × 4 B = 4.68 GB in float32 (VAR_GUIDE §27.1).

[2] Measured per 100,000 trades by VarTierBenchmark (VAR_GUIDE §27.3) and scaled linearly × 23 (2.3M ÷ 100,000): hot file 237.7 MB → 5.47 GB; heap 80 MB an open date → 1.84 GB; first load 0.8–1.2 s → 18–28 s; reopen 23 ms → 0.5 s.

[3] Whole bank: 4.68 GB of float32 rows ÷ 60 GB/s (measured, VAR_GUIDE §27.3) = 78 ms; cold ÷ 3 GB/s (NVMe, the guide's basis) = 1.6 s. A node: 0.48 ms, measured, from its aggregate, whatever its size.

[4] Currencies: 250 MB a currency per 100,000 trades (the guide's extrapolation of 21.4 MB measured on 8,566, VAR_GUIDE §15.11) × 23 × 4 = 23 GB a date; × 5 retained dates = 115 GB; build 4.3 s × 23 = 99 s.

[5] USD hot files: 5.47 GB × 60 dates kept (drishti.var.tier.disk-dates) = 328 GB of local disk.

[6] Lake: 9.37 GB × 0.75 (Parquet, the design's estimate) × 252 business dates = 1.77 TB a year; × 7 years = 12.4 TB.

[7] Portfolios: 15,000 × 130 positions each (ASSUMPTION) = 1.95M rows × 2,036 B = 3.97 GB; ÷ 60 GB/s = 0.07 s, ÷ 5 GB/s scattered (the design's figure) = 0.8 s; stored: 15,000 × 509 × 8 B = 61 MB.

[8] Currency totals: 510 × 8 B × 5,700 nodes (5,000 MGMT and LEGAL, ASSUMPTION, + 700 limit letters) × 10 trade currencies (ASSUMPTION) = 233 MB; a changed position touches ≈ 11 nodes (three trees) × 5 books = 55 additions.

# How the numbers were derived (continued)

## 10-day and stressed VaR

[1] Width: 509 (1-day, 2 years) + 509 (10-day overlapping: 509 − 9 = 500, sized as 509) + 250 + 250 (stressed 1-day and 10-day over 12 months, ASSUMPTION) = 1,518 values a position; ÷ 509 = × 2.98. Method (b) is the bank's to confirm.

[2] Vectors: 2.3M × 1,518 × 8 B = 27.93 GB a date (1-day 9.37); float32 half: 13.97 GB (4.68).

[3] USD hot file: 5.47 GB (237.7 MB per 100,000 measured × 23) × 2.98 = 16.30 GB.

[4] Heap of an open date: 1.84 GB (80 MB per 100,000 × 23) + its aggregates 0.20 GB (8.6 MB per 100,000 × 23, VAR_GUIDE §27.3) × (2.98 − 1) = 2.23 GB.

[5] First open: lake read 18–70 s (the guide's 10–40 s for ≈ 4 GB of Parquet, scaled to 7 GB) × 2.98 = 54–209 s; build 0.8–1.2 s per 100,000 × 23 = 18–28 s × 2.98 = 54–84 s.

[6] Other currency: 1-day built ahead 5.75 GB (250 MB per 100,000 × 23); its 10-day and stressed books on first ask (ASSUMPTION): 5.75 × (2.98 − 1) = 11.4 GB.

[7] Per server: NVMe and RAM as on the processes-and-memory derivations (458 and 1,183 GB; 76 and 151 GB); lake over 7 years as on the storage derivations (16.1 and 48.2 TB).

[8] Option (a), √10 × 1-day: width 509 + 250 = 759 (× 1.49); vectors over 7 years 12.39 TB × 1.49 = 18.5 TB. Sums: a 509-wide set 4.68 GB ÷ 60 GB/s = 78 ms; a 250-wide set 2.3 GB → 38 ms.

## Processes and memory

[1] Heap: two USD dates with all four sets 2 × 2.23 + two other-currency 1-day books 2 × 1.84 (ASSUMPTION) + a load's sort 2 + the rest of the server 4 (ASSUMPTION) = 14.14 GB × 1.5 (generational ZGC) = 21.2 GB need; ÷ 0.61 = 34.8 → -Xmx36g. 1-day: 13.36 × 1.5 = 20.0 GB.

[2] Memory budget (drishti.var.memory-budget-mb, the float32 copy of one date): 2.3M × 1,518 × 4 B = 13,966 MB → 14,000 need; ÷ 0.61 = 22,894 → 23,000 MB; large 2× → 46,000.

[3] Page cache: two USD dates 2 × 16.30 + four 1-day currency books 23 + two currencies' 10-day and stressed books 2 × 11.4 (ASSUMPTION) = 78.4 GB (1-day 33.9). A load: spill 27.93 + new hot file 16.30 = 44.2 GB (1-day 14.8). OS and native 7 GB (ASSUMPTION).

[4] RAM: 21.2 + 78.4 + 44.2 + 7 = 150.9 GB need (1-day 75.8); ÷ 0.61 = 247 → 256 GB. Headroom (39%) is the deck's rule: the projected peak uses at most 61% of what is provisioned.

[5] Local NVMe: USD hot files 16.30 × 60 + 1-day currency books 23 × 5 dates + 4 × 11.4 + a load 44.2 = 1,183 GB; ÷ 0.61 = 1,940 → 2 × 1.9 TB; large 2× → 4 × 1.425 TB.

[6] CPU: 24 threads reached ≈ 60 GB/s (measured, VAR_GUIDE §27.2) + 8 for a load, currency builds, intraday and requests (ASSUMPTION) = 32; ÷ 0.61 = 52 → 64. All four sets of the bank: 78 + 78 + 38 + 38 ≈ 233 ms.

[7] Consoles: 40 pages/s a worker (measured, SCALE_BENCHMARK §8); 50 of 200 users at once, a page every 10 s, peaks 5× (ASSUMPTION) = 25 pages/s ÷ 0.61 = 41 against one host's 4 × 40 = 160.

[8] Large (2× positions): heap 36.4 ÷ 0.61 → 60g; RAM 291 ÷ 0.61 = 477 → 512 GB; vCPU 48 ÷ 0.61 → 96.

# How the numbers were derived (continued)

## Storage over 7 years

[1] Vectors: 27.93 GB a date × 0.75 (Parquet, the design's estimate) × 1,764 dates (7 × 252) = 36.95 TB (1-day 12.39); all four = 1-day × 2.98.

[2] Row columns: ≈ 100 B a row (ASSUMPTION) × 2.3M × 1,764 = 0.41 TB a set × 4 = 1.62 TB. Scenario PVs kept as well (ASSUMPTION, upper case): the vectors again, + 37.0 TB.

[3] Trades: 1.5 KB a trade a day (measured, SCALE_BENCHMARK §8) × 2.3M × 1,764 = 6.09 TB. Membership: 40.8 B a position (measured on the demo: 349,855 B ÷ 8,566) × 2.3M × 1,764 = 0.17 TB.

[4] Official results: 27,527 B capture + 352 B record + 321 B published row (measured on the demo) + ≈ 1 KB JSONL (ASSUMPTION) = 29.2 KB × 5,700 nodes (ASSUMPTION) × 1,764 = 0.29 TB; × 2.98 = 0.88 TB.

[5] Intraday: 230,000 positions changed a day (10%, ASSUMPTION) × 1,518 × 8 B × 0.75 = 9.1 KB → 2.09 GB a day × 1,764 × 2 (compaction keeps the old files) = 7.39 TB (1-day 2.48). Restatements: 2% of dates (ASSUMPTION) × 20.9 GB × 1,764 = 0.74 TB.

[6] Delta checkpoints: one a day ≈ 0.14 TB over 7 years (the design's estimate) × 2.98 = 0.42 TB; one every 10 commits at 120 commits a day: 4.9 TB (upper case). Scenario FX under 1 GB; Iceberg metadata, if chosen, ≈ 12 GB (the product owner's estimate).

[7] Need, base: the rows above = 48.2 TB (1-day 16.1); ÷ 0.61 = 79 TB; a DR copy doubles it: 158 TB. Upper: + PVs 37.0 + trades 6.1 + frequent checkpoints 4.5 = 95.7 TB; ÷ 0.61 = 157; DR 314.

[8] 250-day window: 0.66 × the 1,518 values (1,000 kept): float32 14.0 → 9.2 GB; vectors over 7 years 37.0 → 24.3 TB, only if the bank sends 250-day sets.

[9] Local NVMe a server: USD hot files 16.3 × 60 = 978 + currency books 161 + a load 44 = 1,183 GB; ÷ 0.61 = 1,940 → 2 × 1.9 TB. Rebuilt from the lake.

## A sharded cluster

[1] Six owners, two replicas: each owner holds 2 ÷ 6 = 1/3 of what divides: page cache 78.4 × 1/3 = 26.1 GB; a load 44.2 × 1/3 = 14.7 GB; NVMe 1,183 × 1/3 = 394 GB, ÷ 0.61 = 647 → ~650 GB.

[2] The heap does not divide: every member keeps the date's skeleton for all 2.3M positions (VAR_SHARDING §13): 21.2 GB, the single server's; ÷ 0.61 = 34.8 → -Xmx36g.

[3] An owner's RAM: 21.2 + 26.1 + 14.7 + 7 = 69.1 GB need; ÷ 0.61 = 113 → 128 GB. vCPU 16 an owner (ASSUMPTION; 96 in all).

[4] Coordinators (coordinator-only) and consoles: ASSUMPTION; a coordinator keeps the skeleton without the rows (16–20 GB heap).

[5] 1-day only, 4 owners: heap (2 × 1.84 + 2 + 4) × 1.5 = 14.5 → 16 GB; cache 33.9 × 2/4 = 17.0; load 14.8 × 2/4 = 7.4; OS 7: 47.4 GB; ÷ 0.61 = 78 → 96 GB; heap ÷ 0.61 → 24g. Six owners, 1-day: 16 + 33.9/3 + 14.8/3 + 7 = 39 GB.

[6] Minimum start, 3 owners × 2 replicas (2/3 each): 21.2 + (78.4 + 44.2) × 2/3 + 7 = 110 GB; ÷ 0.61 = 180 → 192 GB; with 1 replica a third each: 69 → 128 GB.

[7] Portfolios: 15,000 × 130 positions (ASSUMPTION) = 1.95M ids × 16 B = 31 MB. The nightly official run over them, "minutes", is a judgement, not computed: the 200K benchmark will measure it.

# How the numbers were derived (continued)

## Concurrency and throughput

[1] Measured: SCALE_STRATEGY §4.4, round 3 (24 cores, -Xmx8g, 1.16M trades a day): 133 / 141 / 122 req/s at 10 / 50 / 100 users, 0 failures, single trade p95 0.08 s at 10 users; §12: one console 69 req/s at 50 users; VAR_GUIDE §27.2: a node under 1 ms, 2.6 ms, 52 ms, 58 ms.

[2] Servers: 140 req/s on 24 cores, scaled by vCPU: × 16/24 = 93 an owner, × 8/24 = 47 a coordinator; 6 × 93 + 2 × 47 = 653 → ~650 peak; half kept free (ASSUMPTION) = 327 → ~330 sustained.

[3] Whole-bank filters: 4.68 GB of float32 rows each (1-day); 6 owners × 60 × 16/24 = 40 GB/s = 240 GB/s → 51 a second at most; 10–25 quoted (20–50% of it, the rest of the load sharing memory bandwidth: ASSUMPTION). Nodes come from totals: thousands a second.

[4] Consoles: 2 × 69 req/s (measured) = 138 → ~100–140 page loads a second.

[5] Active people: the consoles bind first: 138 req/s ÷ 0.3–0.5 req/s a busy person (a page every 10 s, 3–5 calls, ASSUMPTION) = 276–460 → ~300–500. The servers' ~330 sustained would carry more.

[6] Named users: active ÷ 10–20% active at once (ASSUMPTION): 300 ÷ 0.2 = 1,500 to 500 ÷ 0.1 = 5,000.

## Hardware plan

[1] VaR servers: 4, one machine type (official EOD, intraday writer, two UI/API replicas); vCPU, RAM, heap, memory budget and NVMe as on the processes-and-memory derivations: provisioned = need ÷ 0.61, up to the next size; large = 2× positions.

[2] Consoles: 25 pages/s at peak (ASSUMPTION: 50 of 200 users, a page every 10 s, peaks 5×) ÷ 0.61 = 41 against 160 a host.

[3] Lake: 48.2 TB need ÷ 0.61 = 79 TB; × 2 with a DR copy; large × 2. Shared state: ≈ 300 GB as built (config, Sutras, each node's data folder; ASSUMPTION) ÷ 0.61 = 0.49 → 0.5 TB.

[4] Network: a first open reads 27.93 × 0.75 = 20.9 GB of Parquet at ≤ 0.4 GB/s (decoding, the guide's 10–40 s for 4 GB) = 3.2 Gbit/s ÷ 0.61 = 5.2 → ≥ 10 Gbit/s.

[5] Intraday: 2.09 GB a day (storage derivations) ÷ 120 five-minute windows = 17.5 MB a window, < 1 MB/s a server.

[6] Kafka: 230,000 records a day × 24 KB (1,518 numbers as JSON, ASSUMPTION; the guide: 4–8 KB for 509) = 5.5 GB a day × 7 days = 38.6 GB ÷ 0.61 = 63 → 64 GB a replica.

[7] Sharding beyond ≈ 5M positions: (151 − 13 fixed) GB ÷ 2.3M ÷ 0.61 = 98 GB of RAM a million positions, so 512 GB holds ≈ 5.2M; a first open of 2–5 min at 2.3M takes 4–10 min at 5M.

## AWS: the VaR tier

[1] A, ECS on EC2 capacity: 151 GB need ÷ 0.61 = 247 → m7i.16xlarge (64 vCPU, 256 GiB); EBS gp3 2 TB (1,183 ÷ 0.61 = 1,940 GB).

[2] 1-day only on one Fargate task: 75.8 GB, 32 vCPU need; ÷ 0.61 = 124 GB, 52 vCPU: the 32 vCPU · 244 GB task holds the memory (31%) but keeps the CPU headroom only if the peak is ≤ 32 × 0.61 = 19.5 vCPU. Full case: 151 ÷ 244 = 62%, over 61%.

[3] B, √10 for 10-day: width 759 (× 1.49): heap, page cache, load and OS = 94 GB; ÷ 0.61 = 155 GB ≤ 244 GB; 1.1 TB gp3.

[4] C, 8 shards: (151 − 6 heap base − 7 OS) ÷ 8 + 6 + 3 = 26 GB ÷ 0.61 = 43 → an 8 vCPU, 44 GB task; CPU 32 ÷ 8 + 1 = 5 ÷ 0.61 = 8.2 → 8; gp3 1,183 ÷ 8 ÷ 0.61 = 242 → 250 GB.

[5] Fargate and EBS limits: AWS documentation, read 2026-10-27 (32 vCPU with up to 244 GB; one EBS volume a task; gp3 up to 64 TiB, 80,000 IOPS, 2,000 MiB/s).

# How the numbers were derived (continued)

## AWS cost

[1] Prices: AWS public on-demand list prices, example Region ca-central-1, read 2026-10-27 from the Price List files (ASSUMPTION: verify before quoting); 730 hours a month; quantities from the capacity derivations (ESTIMATED).

[2] VaR servers: 4 × m7i.16xlarge $3.5952 an hour × 730 = $10,498 (1-day: c7i.16xlarge $3.1248 → $9,124). EBS gp3 a server: 2,000 GB × $0.088 + 13,000 IOPS × $0.0055 + 1,875 MiB/s × $0.044 = $330 × 4 (1-day 800 GB: $224 × 4).

[3] Consoles: 2 × (4 vCPU × $0.04456 + 8 GB × $0.004865) × 730 = $317; lake tasks 60 h × (8 × $0.04456 + 32 × $0.004865) = $31.

[4] S3: the lake grows 6.9 TB a year (1-day 2.3); year 7: newest year Standard $0.025 + six older years Standard-IA $0.0138 per GB-month (a lifecycle rule, ASSUMPTION) = $752; DR: the same + $0.02/GB replication = $753.

[5] EFS 500 GB × $0.33 + $25 backup = $190; MSK 3 × $0.2275 × 730 + 300 GB × $0.11 = $531; ALB, NAT and endpoints, transfer, CloudWatch as listed; Secrets Manager and KMS $4, in the total.

[6] Savings Plans (Compute, no upfront, the same price files): 1 year −26.9% EC2, −20.0% Fargate; 3 years −49.5%, −45.0%; applied to EC2 and Fargate only.

[7] VaR tier: A as above $11,818; B 4 × (32 × $0.04456 + 244 × $0.004865 = $2.613 an hour) × 730 + 4 × 1.1 TB gp3 = $8,633; C 16 tasks × $0.5705 × 730 + 16 × $38.50 gp3 + the intraday task $93 = $7,373.

[8] C scale-in: second replicas 12 h × 22 days = 264 of 730 hours (ASSUMPTION) = $5,050; 3-year $2,966; second replicas on Spot at 60% off (ASSUMPTION) = $2,785. One full copy: Fargate 8 × ($416 + $38.50) = $3,640 against EC2 $2,954.

RELATED DOCUMENTS

# Where to read more

## Companion decks

**Drishti: Design and Evidence**
what Drishti is, its design, editions, BI, AI, security and the evidence

**Drishti on AWS: the lake, Delta or Iceberg**
the AWS architecture in detail: setup, storage, caching, maintenance, cost and runbooks, for either format

**Drishti VaR engine**
this deck: method, inputs, runs, capacity, editions, regulatory reporting, roadmap

| Document | What it holds |
|---|---|
| `docs/guides/VAR_GUIDE.md` | the complete guide: using VaR, the data contract, how the engine stores data |
| `docs/tutorials/25-var-from-the-bank-to-a-trade.md` | VaR from the bank to a trade, in twelve hands-on steps |
| `docs/knowledge/market-risk.md` | the Market risk Knowledge page in the console |
| `docs/architecture/VAR_SHARDING.md` | VaR sharded across servers |
| `docs/architecture/EDITIONS.md` | editions and lenses; Risk, Market, World |
| `docs/admin/EDITIONS_GUIDE.md` | running an edition or a lens |
| `docs/architecture/COMPUTED_ATTRIBUTES.md` | computed on demand, never stored |
| `docs/architecture/REGULATORY_VAR_REPORTING.md` | the Canadian and US reports that need VaR or ES |
| `docs/architecture/IMPLEMENTATION_PLAN.md` | what is built, and what comes next |



Drishti दृष्टि

# Thank you

*See what your data hasn't told you yet.*

ANY DATA. ANY DOMAIN. ONE GRAMMAR.

One engine, every input: PVs or P&L, by trade or by position, any currency, files or Kafka.

Ashutosh Sinha • October 2026 • VaR engine